

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل یازدهم: Outlier detection

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 11.1 Basic concepts
 - 11.1.1 What are outliers?
 - 11.1.2 Types of outliers
 - 11.1.3 Challenges of outlier detection
 - 11.1.4 An overview of outlier detection methods
- 11.2 Statistical approaches
 - 11.2.1 Parametric methods
 - 11.2.2 Nonparametric methods
- 11.3 Proximity-based approaches
 - 11.3.1 Distance-based outlier detection
 - 11.3.2 Density-based outlier detection
- 11.4 Reconstruction-based approaches
 - 11.4.1 Matrix factorization-based methods for numerical data
 - 11.4.2 Pattern-based compression methods for categorical data
- 11.5 Clustering- vs. classification-based approaches
 - 11.5.1 Clustering-based approaches
 - 11.5.2 Classification-based approaches
- 11.6 Mining contextual and collective outliers
 - 11.6.1 Transforming contextual outlier detection to conventional outlier detection
 - 11.6.2 Modeling normal behavior with respect to contexts
 - 11.6.3 Mining collective outliers
- 11.7 Outlier detection in high-dimensional data
 - 11.7.1 Extending conventional outlier detection
 - 11.7.2 Finding outliers in subspaces
 - 11.7.3 Outlier detection ensemble
 - 11.7.4 Taming high dimensionality by deep learning
 - 11.7.5 Modeling high-dimensional outliers

Section 11.1

11.1.2. Types of outliers

انواع داده‌های پرت (Types of Outliers) ⚠

داده‌های پرت (Outliers) در یک مجموعه داده می‌توانند اشکال مختلفی داشته باشند و درک انواع آن‌ها برای تشخیص مؤثر داده‌های پرت حیاتی است. به طور کلی، داده‌های پرت را می‌توان به انواع زیر دسته‌بندی کرد:

- داده‌های پرت کلی (Global Outliers):** داده‌های پرت کلی، نقاط داده‌ای هستند که به طور قابل توجهی از الگوی کلی کل مجموعه داده منحرف می‌شوند. آن‌ها با مقادیر افراطی که از تمایل مرکزی داده‌ها دور هستند، مشخص می‌شوند.
- داده‌های پرت زمینه‌ای (Contextual Outliers):** داده‌های پرت زمینه‌ای، نقاط داده‌ای هستند که در یک زمینه یا زیرمجموعه خاصی از داده‌ها غیرعادی هستند. شناسایی داده‌های پرت زمینه‌ای نیازمند در نظر گرفتن ویژگی‌های محلی

داده‌ها است.

- **داده‌های پرت جمعی (Collective Outliers):** داده‌های پرت جمعی، که به آن‌ها داده‌های پرت خوشه‌ای (cluster outliers) یا زیرجمعیت‌ها (subpopulations) نیز گفته می‌شود، به گروه‌هایی از نقاط داده اشاره دارند که با هم یک الگوی ناهنجار را تشکیل می‌دهند. تشخیص داده‌های پرت جمعی شامل شناسایی ساختارهای غیرعادی در داخل مجموعه داده است.
- **داده‌های پرت ویژگی (Attribute Outliers):** داده‌های پرت ویژگی، مقادیر افراطی را در یک یا چند ویژگی (attributes or features) از خود نشان می‌دهند. این داده‌های پرت می‌توانند با بررسی ویژگی‌های فردی به طور مستقل شناسایی شوند.

درک انواع داده‌های پرت برای انتخاب تکنیک‌های تشخیص داده پرت مناسب و تفسیر نتایج در زمینه ویژگی‌های داده‌های خاص ضروری است.

اکنون، بیا **یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تشخیص داده پرت با استفاده از الگوریتم Isolation Forest ارائه دهیم:**

In []: *Isolation Forest آموزش: تشخیص نقاط پرت با استفاده از الگوریتم*

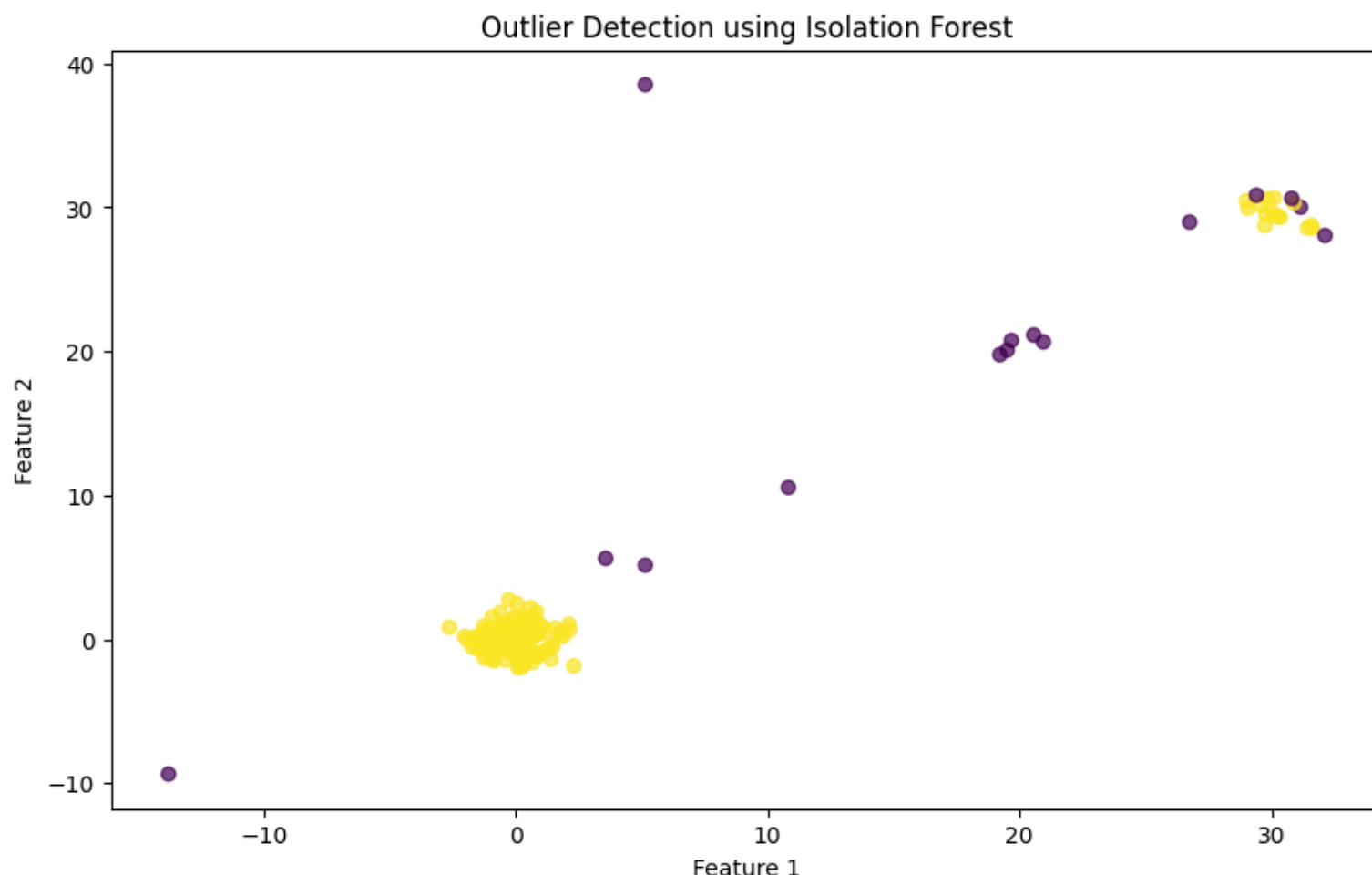
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest

# تولید داده‌های مصنوعی شامل انواع نقاط پرت 1
np.random.seed(42)
data_normal = np.random.normal(loc=0, scale=1, size=(100, 2))          # داده‌های نرمال
data_global_outliers = np.random.normal(loc=0, scale=10, size=(5, 2))  # نقاط پرت سراسری
data_contextual_outliers = np.random.normal(loc=20, scale=1, size=(5, 2)) # نقاط پرت زمینه‌ای
data_collector_outliers_1 = np.random.normal(loc=0, scale=1, size=(20, 2)) # خوشه اول پرت جمعی
data_collector_outliers_2 = np.random.normal(loc=30, scale=1, size=(20, 2)) # خوشه دوم پرت جمعی

# ترکیب همه داده‌ها در یک مجموعه 2
data = np.vstack([
    data_normal,
    data_global_outliers,
    data_contextual_outliers,
    data_collector_outliers_1,
    data_collector_outliers_2
])

# برای تشخیص نقاط پرت Isolation Forest اعمال الگوریتم 3
isolation_forest = IsolationForest(contamination=0.1, random_state=42)
labels = isolation_forest.fit_predict(data) # برچسب‌های 1- (پرت) و 1 (نرمال)

# نمایش تصویری نتایج تشخیص 4
plt.figure(figsize=(10, 6))
plt.scatter(data[:, 0], data[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.title('Outlier Detection using Isolation Forest')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، الگوریتم **Isolation Forest** برای تشخیص انواع نقاط پرت در داده‌های دوبعدی مصنوعی استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های نرمال و انواع نقاط پرت (سراسری، زمینه‌ای، جمعی) تولید شده‌اند.
 - داده‌ها با استفاده از `vstack` ترکیب شده‌اند تا یک مجموعه کامل تشکیل شود.
 - الگوریتم **Isolation Forest** با پارامتر `contamination=0.1` آموزش دیده و برچسب‌های 1- (پرت) و 1 (نرمال) تولید کرده است.
 - نتایج با استفاده از `scatter` در فضای دوبعدی نمایش داده شده‌اند تا توزیع نقاط پرت قابل مشاهده باشد.
- این مثال پایه‌ای مناسب برای درک نحوه عملکرد الگوریتم‌های تشخیص ناهنجاری در داده‌های چندبعدی است. الگوریتم Isolation Forest به‌ویژه برای داده‌های بزرگ و با ابعاد بالا بسیار مؤثر و مقیاس‌پذیر است.

Section 11.2

11.2.1. Parametric methods

روش‌های پارامتریک برای تشخیص داده پرت (Parametric methods for outlier detection)

روش‌های پارامتریک برای تشخیص داده پرت (outlier detection) فرض می‌کنند که توزیع زیربنایی داده‌ها را می‌توان با یک مدل پارامتریک خاص (مانند توزیع گوسی یا Gaussian) مشخص کرد. این روش‌ها برای شناسایی نقاط داده‌ای که به طور قابل توجهی از توزیع مورد انتظار منحرف می‌شوند، به معیارهای آماری و آزمون فرضیه (hypothesis testing) متکی هستند.

نکات کلیدی درباره روش‌های پارامتریک:

1. **فرضیه توزیع:** روش‌های پارامتریک فرض می‌کنند که داده‌ها از یک توزیع شناخته شده پیروی می‌کنند. توزیع‌های متداول شامل توزیع نرمال (normal) یا گوسی هستند.

2. **امتیاز Z:** امتیاز Z یک روش پارامتریک رایج است که اندازه‌گیری می‌کند یک نقطه داده چند انحراف معیار (standard deviations) از میانگین فاصله دارد. نقاط داده با امتیاز Z مطلق بالا، داده پرت در نظر گرفته می‌شوند.

3. **آزمون فرضیه:** روش‌های پارامتریک اغلب شامل آزمون فرضیه برای ارزیابی اینکه آیا یک نقطه داده یا مجموعه‌ای از نقاط داده به طور قابل توجهی از توزیع مورد انتظار منحرف می‌شوند، می‌باشند. آزمون‌های رایج شامل t-test و chi-square test هستند.

4. **رویکردهای مبتنی بر مدل:** برخی از روش‌های پارامتریک، مدل‌های صریحی از توزیع داده‌ها می‌سازند و داده‌های پرت را بر اساس احتمال داده‌های مشاهده شده تحت مدل شناسایی می‌کنند.

5. **حساسیت به فرضیه:** اثربخشی روش‌های پارامتریک به دقت فرضیه‌های توزیعی بستگی دارد. اگر داده‌ها به طور قابل توجهی از توزیع فرضی منحرف شوند، روش‌های پارامتریک ممکن است نتایج نادرستی ارائه دهند.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تشخیص داده پرت با استفاده از امتیاز Z ارائه دهیم:

In []: Z-Score آموزش: تشخیص نقاط پرت با استفاده از #

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import zscore

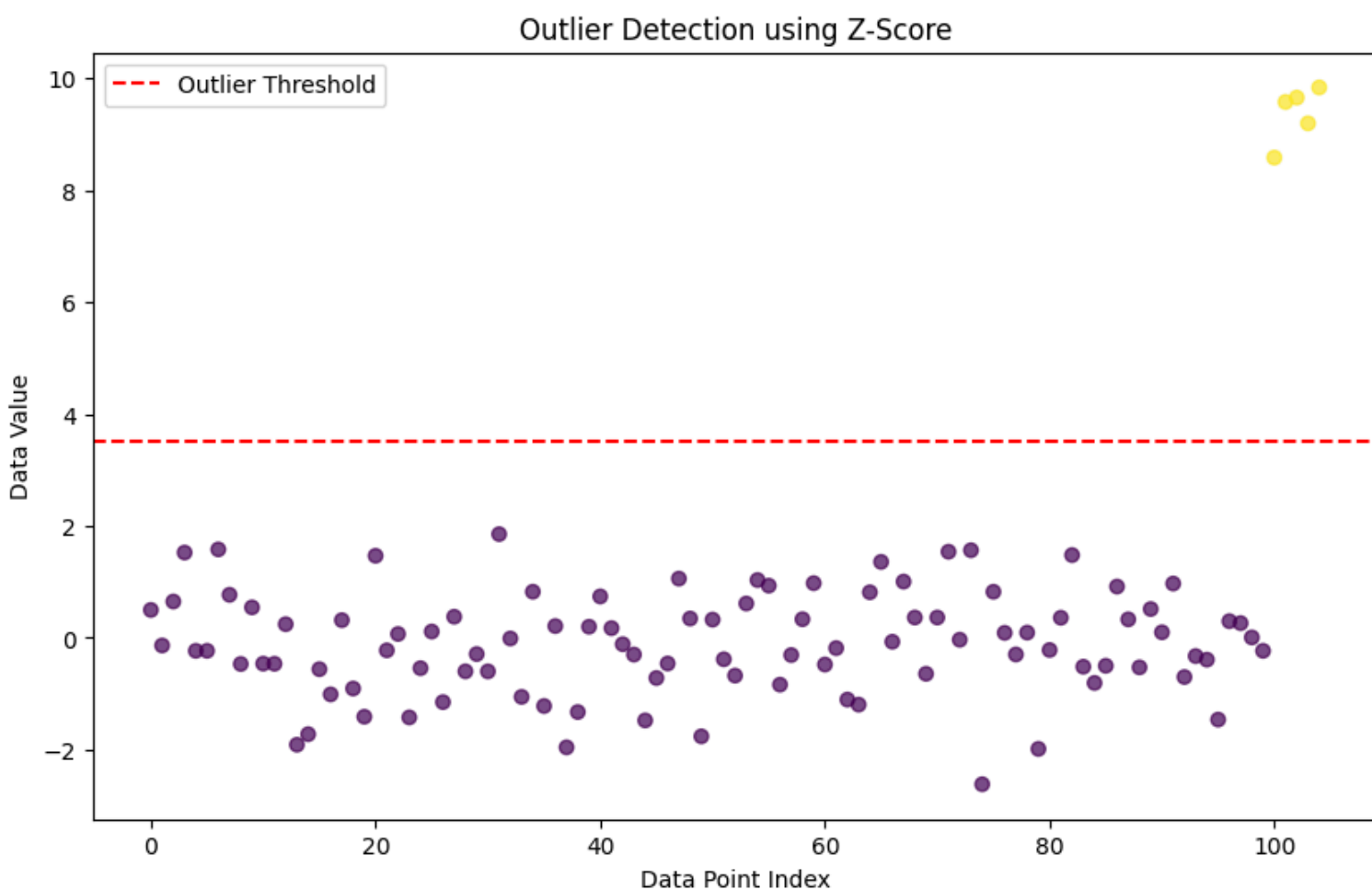
# تولید داده‌های مصنوعی شامل داده‌های نرمال و نقاط پرت 1
np.random.seed(42)
data_normal = np.random.normal(loc=0, scale=1, size=100) # داده‌های نرمال
data_outliers = np.random.normal(loc=10, scale=1, size=5) # نقاط پرت شدید

# ترکیب داده‌های نرمال و پرت در یک آرایه 2
data = np.concatenate([data_normal, data_outliers])

# برای هر نقطه Z-Score محاسبه 3
z_scores = zscore(data)

# تعیین آستانه برای تشخیص نقاط پرت 4
threshold = 3.5
outliers = np.abs(z_scores) > threshold # بزرگ‌تر از آستانه Z-Score نقاطی با

# نمایش تصویری نتایج تشخیص 5
plt.figure(figsize=(10, 6))
plt.scatter(range(len(data)), data, c=outliers, cmap='viridis', alpha=0.7)
plt.axhline(y=threshold, color='r', linestyle='--', label='Outlier Threshold')
plt.title('Outlier Detection using Z-Score')
plt.xlabel('Data Point Index')
plt.ylabel('Data Value')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از روش آماری **Z-Score** برای تشخیص نقاط پرت در داده‌های عددی استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های نرمال و نقاط پرت مصنوعی تولید شده‌اند و با هم ترکیب شده‌اند.
- برای هر نقطه، مقدار Z-Score محاسبه شده تا فاصله آن از میانگین مشخص شود.
- نقاطی که مقدار Z-Score آن‌ها از آستانه تعیین‌شده (۳.۵) بیشتر باشد، به‌عنوان پرت شناسایی شده‌اند.
- نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا نقاط پرت به‌صورت رنگی قابل تشخیص باشند.

روش Z-Score برای داده‌های تک‌بعدی و توزیع‌های نرمال بسیار مؤثر است، اما در داده‌های چندبعدی یا غیرنرمال بهتر است از روش‌هایی مانند **Isolation Forest** یا **LOF** استفاده شود.

11.2.2. Nonparametric methods

روش‌های ناپارامتریک برای تشخیص داده پرت (Nonparametric methods for outlier detection)

روش‌های ناپارامتریک برای تشخیص داده پرت (outlier detection) فرضیات قوی در مورد توزیع زیربنایی داده‌ها ندارند. این روش‌ها برای شناسایی نقاط داده‌ای که از الگوی مورد انتظار منحرف می‌شوند، به رتبه‌بندی یا سایر تکنیک‌های مستقل از توزیع متکی هستند.

نکات کلیدی درباره روش‌های ناپارامتریک:

- تکنیک‌های مبتنی بر رتبه:** روش‌های ناپارامتریک اغلب از آمار مبتنی بر رتبه، مانند میانه و دامنه بین چارکی (interquartile range - IQR) استفاده می‌کنند که نسبت به میانگین و انحراف معیار، حساسیت کمتری به مقادیر افراطی دارند.
- رویکردهای مبتنی بر صدک:** داده‌های پرت می‌توانند بر اساس صدک‌ها (percentiles) یا چارک‌های (quantiles) توزیع داده‌ها شناسایی شوند. نقاط داده‌ای که خارج از یک محدوده صدکی خاص قرار می‌گیرند، داده پرت در نظر گرفته می‌شوند.
- تخمین چگالی هسته (KDE):** KDE یک روش ناپارامتریک است که تابع چگالی احتمال داده‌ها را تخمین می‌زند. داده‌های پرت می‌توانند بر اساس نواحی با چگالی کم در توزیع تخمین زده شده، شناسایی شوند.
- فاکتور داده پرت محلی (LOF):** LOF یک روش ناپارامتریک است که انحراف چگالی محلی یک نقطه داده را نسبت به همسایگانش اندازه‌گیری می‌کند. نقاط با چگالی محلی به طور قابل توجهی کمتر، داده پرت در نظر گرفته می‌شوند.
- آزمون‌های مستقل از توزیع:** روش‌های ناپارامتریک اغلب از آزمون‌های آماری استفاده می‌کنند که به فرضیات توزیعی خاصی متکی نیستند. به عنوان مثال، آزمون کولموگوروف-اسمیرنوف (Kolmogorov-Smirnov) ارزیابی می‌کند که آیا یک نمونه از یک توزیع خاص می‌آید یا خیر.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تشخیص داده پرت با استفاده از الگوریتم فاکتور داده پرت محلی (LOF) ارائه دهیم:

```
In [ ]: # Local Outlier Factor (LOF) آموزش: تشخیص نقاط پرت با استفاده از الگوریتم

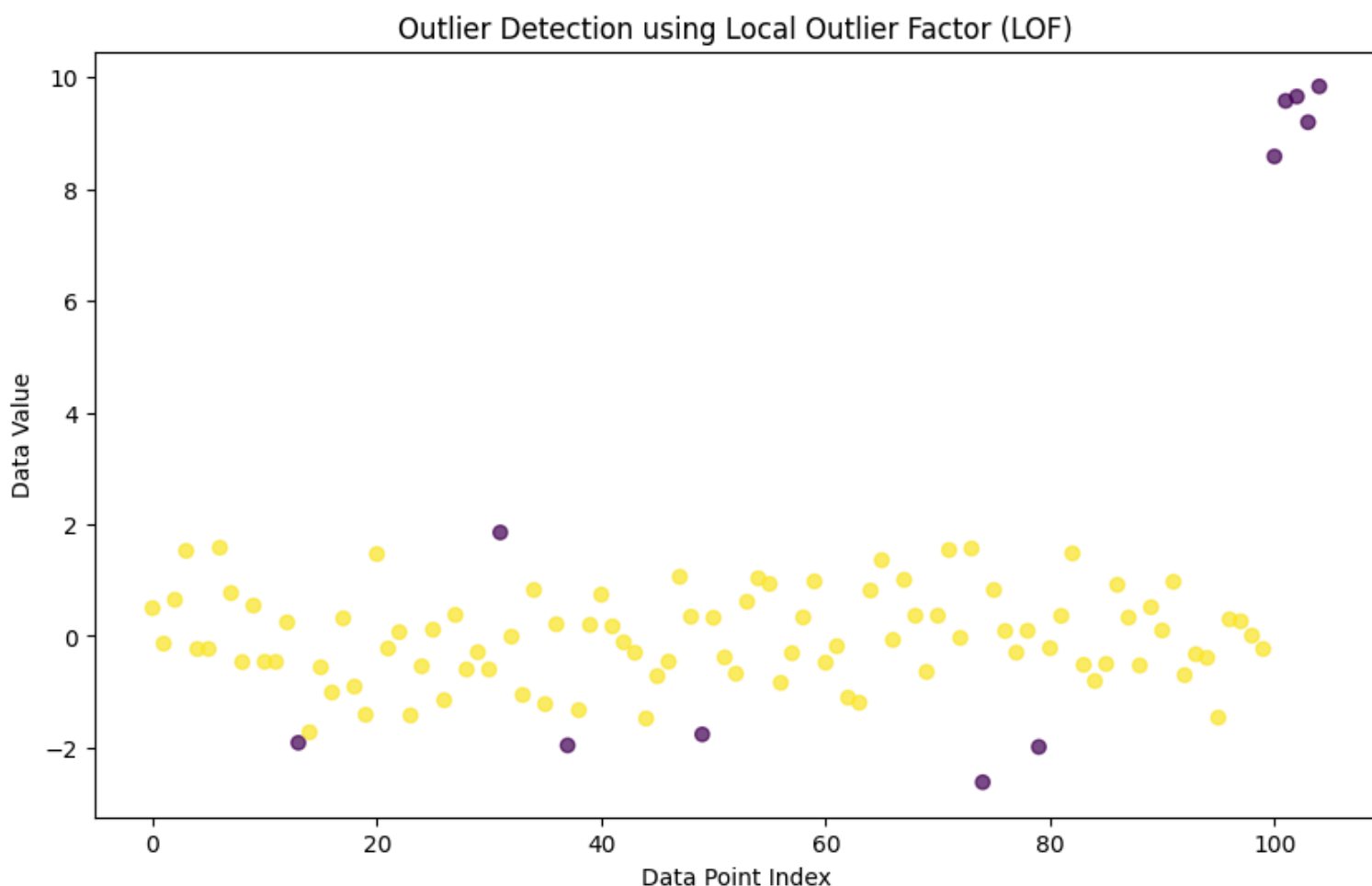
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import LocalOutlierFactor

# 1 تولید داده‌های مصنوعی شامل داده‌های نرمال و نقاط پرت
np.random.seed(42)
data_normal = np.random.normal(loc=0, scale=1, size=100) # داده‌های نرمال
data_outliers = np.random.normal(loc=10, scale=1, size=5) # نقاط پرت شدید

# 2 ترکیب داده‌های نرمال و پرت در یک آرایه و تبدیل به شکل مناسب
data = np.concatenate([data_normal, data_outliers]).reshape(-1, 1)

# 3 برای تشخیص نقاط پرت LOF اعمال الگوریتم
lof = LocalOutlierFactor(contamination=0.1)
labels = lof.fit_predict(data) # 1 (نرمال) و 1- (پرت) برچسب‌های

# 4 نمایش تصویری نتایج تشخیص
plt.figure(figsize=(10, 6))
plt.scatter(range(len(data)), data, c=labels, cmap='viridis', alpha=0.7)
plt.title('Outlier Detection using Local Outlier Factor (LOF)')
plt.xlabel('Data Point Index')
plt.ylabel('Data Value')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، الگوریتم **Local Outlier Factor (LOF)** برای تشخیص نقاط پرت در داده‌های عددی تک‌بعدی استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های نرمال و نقاط پرت مصنوعی تولید شده‌اند و با هم ترکیب شده‌اند.
- داده‌ها به شکل مناسب برای ورودی الگوریتم LOF تبدیل شده‌اند (آرایه دوبعدی).
- الگوریتم LOF با پارامتر `contamination=0.1` آموزش دیده و برچسب‌های 1- (پرت) و 1 (نرمال) تولید کرده است.
- نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا نقاط پرت به‌صورت رنگی قابل تشخیص باشند.

الگوریتم LOF با مقایسه چگالی محلی هر نقطه نسبت به همسایگانش، نقاط پرت را شناسایی می‌کند. این روش برای داده‌های غیرخطی و توزیع‌های پیچیده بسیار مؤثر است.

Section 11.3

11.3.1. Distance-based outlier detection

تشخیص داده پرت مبتنی بر فاصله (Distance-based outlier detection)

روش‌های تشخیص داده پرت مبتنی بر فاصله، داده‌های پرت را با اندازه‌گیری عدم تشابه یا فاصله بین نقاط داده شناسایی می‌کنند. این روش‌ها فرض می‌کنند که داده‌های پرت اغلب ایزوله هستند یا از اکثریت نقاط داده فاصله زیادی دارند، که منجر به فواصل بزرگ‌تر می‌شود.

نکات کلیدی درباره تشخیص داده پرت مبتنی بر فاصله:

- فاصله اقلیدسی:** فاصله اقلیدسی بین نقاط داده یک معیار رایج است که در تشخیص داده پرت مبتنی بر فاصله استفاده می‌شود. این معیار فاصله خط مستقیم بین دو نقطه در فضای ویژگی (feature space) را محاسبه می‌کند.
- فاصله ماحالانوبیس:** فاصله ماحالانوبیس همبستگی بین ویژگی‌ها را در نظر می‌گیرد و یک معیار دقیق‌تر از فاصله در حضور ویژگی‌های همبسته ارائه می‌دهد.
- رویکردهای نزدیک‌ترین همسایه:** تشخیص داده پرت مبتنی بر فاصله اغلب شامل مقایسه فواصل نقاط داده با k نزدیک‌ترین همسایه‌شان است. داده‌های پرت به عنوان نقاطی با فواصل غیرعادی بزرگ به همسایگان خود شناسایی می‌شوند.
- امتیازهای داده پرت:** امتیازهای داده پرت بر اساس فواصل محاسبه می‌شوند و یک آستانه برای تمایز داده‌های پرت از نقاط داده عادی تعیین می‌گردد.
- مقاوم در برابر توزیع:** روش‌های مبتنی بر فاصله اغلب در برابر توزیع زیربنایی داده‌ها مقاوم هستند که آن‌ها را برای انواع مختلفی از مجموعه‌های داده مناسب می‌سازد.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تشخیص داده پرت با استفاده از الگوریتم Isolation Forest که یک روش مبتنی بر فاصله است، ارائه دهیم:

In []: *# (مبتنی بر فاصله) Isolation Forest آموزش: تشخیص نقاط پرت با استفاده از الگوریتم*

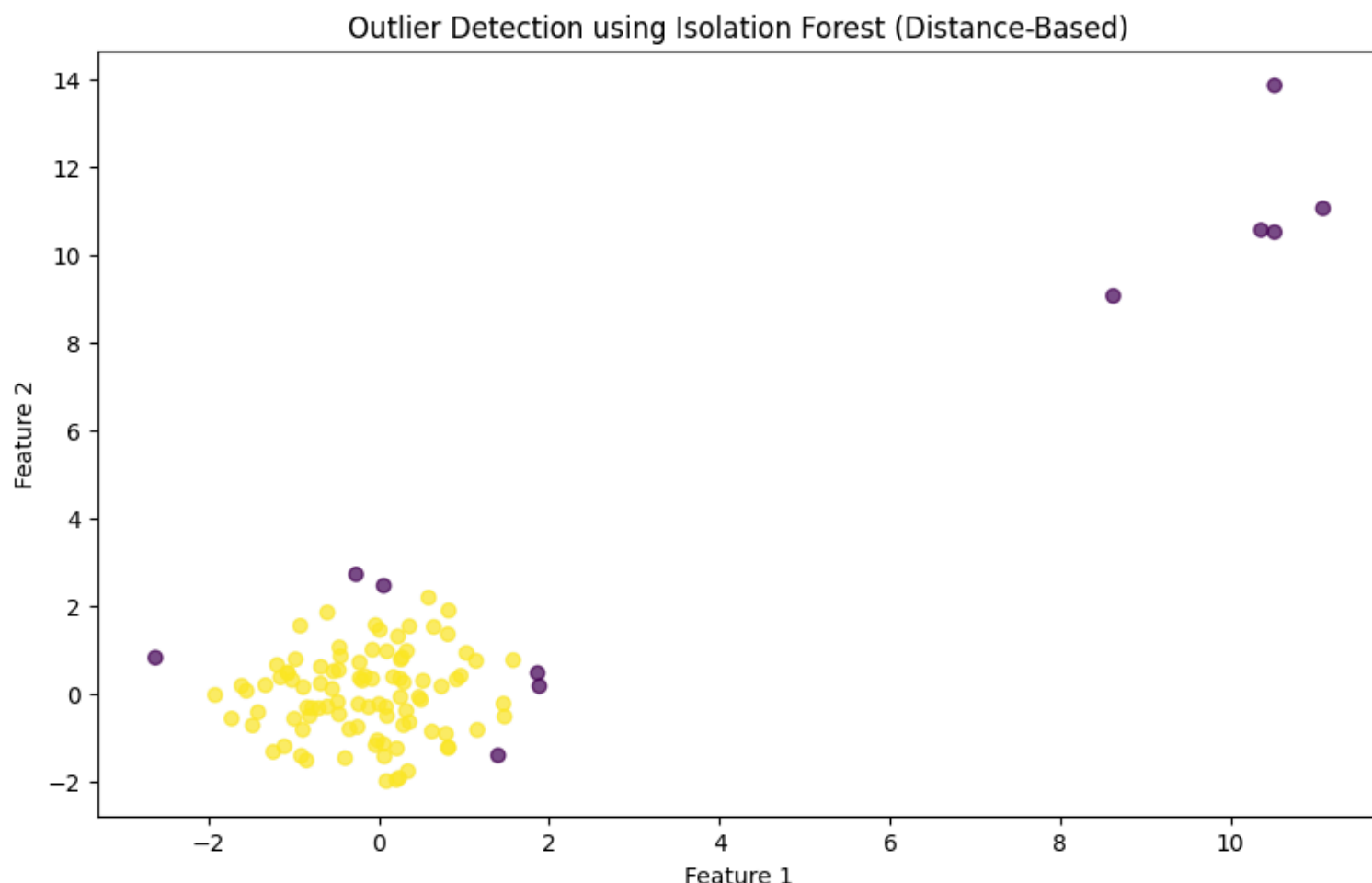
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest

# 1 تولید داده‌های مصنوعی شامل داده‌های نرمال و نقاط پرت
np.random.seed(42)
data_normal = np.random.normal(loc=0, scale=1, size=(100, 2)) # داده‌های نرمال
data_outliers = np.random.normal(loc=10, scale=1, size=(5, 2)) # نقاط پرت شدید

# 2 ترکیب داده‌های نرمال و پرت در یک مجموعه
data = np.vstack([data_normal, data_outliers])

# 3 برای تشخیص نقاط پرت Isolation Forest اعمال الگوریتم
isolation_forest = IsolationForest(contamination=0.1, random_state=42)
labels = isolation_forest.fit_predict(data) # (نرمال) 1 و (پرت) 1- برچسب‌های

# 4 نمایش تصویری نتایج تشخیص
plt.figure(figsize=(10, 6))
plt.scatter(data[:, 0], data[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.title('Outlier Detection using Isolation Forest (Distance-Based)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از الگوریتم **Isolation Forest** برای تشخیص نقاط پرت در داده‌های دوبعدی استفاده شده است. این الگوریتم بر اساس فاصله و جداسازی تصادفی نمونه‌ها عمل می‌کند:

- ابتدا داده‌های نرمال و نقاط پرت مصنوعی تولید شده‌اند و با هم ترکیب شده‌اند.
- الگوریتم Isolation Forest با پارامتر `contamination=0.1` آموزش دیده و نقاط پرت را با برچسب 1 مشخص کرده است.
- نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا نقاط پرت به صورت رنگی قابل تشخیص باشند.

الگوریتم Isolation Forest برای داده‌های بزرگ و چندبعدی بسیار مؤثر است و برخلاف روش‌های آماری، نیازی به فرض توزیع خاص ندارد. این مثال پایه‌ای مناسب برای درک نحوه عملکرد آن در فضای دوبعدی است.

11.3.2. Density-based outlier detection

تشخیص داده پرت مبتنی بر چگالی (Density-based outlier detection)

روش‌های تشخیص داده پرت مبتنی بر چگالی، داده‌های پرت را بر اساس چگالی محلی نقاط داده شناسایی می‌کنند. این روش‌ها فرض می‌کنند که داده‌های پرت با چگالی محلی به طور قابل توجهی پایین‌تر در مقایسه با همسایگان خود مشخص می‌شوند. رویکردهای مبتنی بر چگالی به ویژه در شناسایی داده‌های پرت در مناطقی با چگالی داده متغیر مؤثر هستند.

نکات کلیدی درباره تشخیص داده پرت مبتنی بر چگالی:

1. **تخمین چگالی محلی:** روش‌های مبتنی بر چگالی، چگالی محلی را در اطراف هر نقطه داده تخمین می‌زنند، که معمولاً با در نظر گرفتن تعداد نقاط همسایه در یک فاصله مشخص انجام می‌شود.

2. رویکردهای مبتنی بر خوشه‌بندی: رویکردهای مبتنی بر چگالی اغلب شامل خوشه‌بندی (clustering) هستند، جایی که مناطق متراکم، خوشه در نظر گرفته می‌شوند و داده‌های پرت، نقاطی هستند که به هیچ خوشه‌ای تعلق ندارند یا به یک خوشه با چگالی پایین تعلق دارند.
3. امتیازهای داده پرت: امتیازهای داده پرت بر اساس چگالی محلی محاسبه می‌شوند و نقاط با چگالی کمتر به عنوان داده‌های پرت بالقوه در نظر گرفته می‌شوند.
4. روش مثال: DBSCAN: DBSCAN یک روش محبوب مبتنی بر چگالی است که خوشه‌ها را بر اساس چگالی داده شناسایی می‌کند و نقاط خارج از این خوشه‌ها را به عنوان داده پرت علامت‌گذاری می‌نماید.
5. تطبیق با الگوهای محلی: روش‌های مبتنی بر چگالی با الگوهای محلی تطبیق‌پذیر هستند و می‌توانند داده‌های پرت را در مناطق با چگالی داده متغیر شناسایی کنند، که آن‌ها را برای مجموعه‌های داده پیچیده مناسب می‌سازد.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تشخیص داده پرت با استفاده از DBSCAN ارائه دهیم:

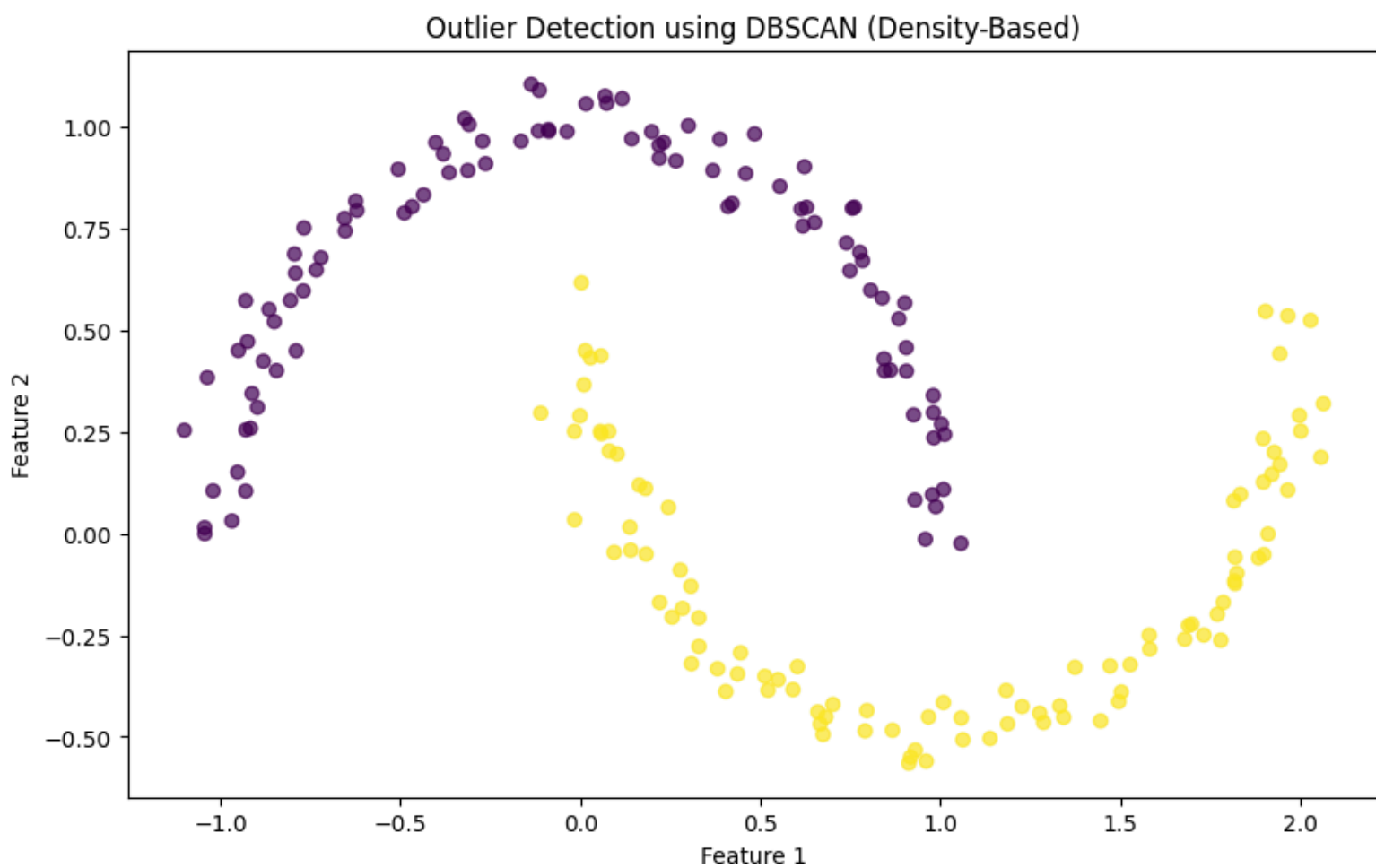
In []: # (مبتنی بر چگالی) DBSCAN آموزش: تشخیص نقاط پرت با استفاده از الگوریتم

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

# تولید داده‌های مصنوعی با ساختار غیرخطی و نویز 1
data, _ = make_moons(n_samples=200, noise=0.05, random_state=42)

# برای تشخیص نقاط پرت و خوشه‌بندی DBSCAN اعمال الگوریتم 2
dbscan = DBSCAN(eps=0.3, min_samples=5)
labels = dbscan.fit_predict(data) # برچسب‌های 1- (پرت) و عددهای خوشه

# نمایش تصویری نتایج تشخیص و خوشه‌بندی 3
plt.figure(figsize=(10, 6))
plt.scatter(data[:, 0], data[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.title('Outlier Detection using DBSCAN (Density-Based)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، الگوریتم DBSCAN برای تشخیص نقاط پرت و خوشه‌بندی داده‌های غیرخطی استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با ساختار `make_moons` تولید شده‌اند که دارای نویز و شکل غیرخطی هستند.
 - الگوریتم DBSCAN با پارامترهای `eps=0.3` و `min_samples=5` اجرا شده و نقاط پرت را با برجسب -1 مشخص کرده است.
 - نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا خوشه‌ها و نقاط پرت به‌صورت رنگی قابل تشخیص باشند.
- DBSCAN یکی از الگوریتم‌های قدرتمند برای تشخیص ناهنجاری در داده‌هایی با ساختار پیچیده و غیرخطی است. این روش بدون نیاز به تعیین تعداد خوشه‌ها، بر اساس چگالی محلی عمل می‌کند و نقاط پرت را به‌صورت طبیعی جدا می‌سازد.

Section 11.4

11.4.1. Matrix factorization–based methods for numerical data

روش‌های مبتنی بر تجزیه ماتریس (Matrix factorization–based methods) برای تشخیص داده پرت

روش‌های مبتنی بر تجزیه ماتریس (Matrix factorization) برای تشخیص داده پرت (outlier detection) شامل تجزیه ماتریس داده اصلی به ماتریس‌های با ابعاد پایین‌تر هستند، با هدف ثبت ساختار زیربنایی داده‌ها. سپس داده‌های پرت بر اساس تفاوت بین داده‌های مشاهده شده و بازسازی شده شناسایی می‌شوند.

نکات کلیدی درباره روش‌های مبتنی بر تجزیه ماتریس:

1. **تجزیه ماتریس:** تکنیک‌های تجزیه ماتریس، ماتریس داده اصلی را به دو یا چند ماتریس با ابعاد پایین‌تر تجزیه می‌کنند. روش‌های رایج شامل تجزیه مقدار منفرد (Singular Value Decomposition - SVD) و تجزیه ماتریس غیرمنفی (Non-Negative Matrix Factorization - NMF) هستند.
2. **خطای بازسازی:** تفاوت بین ماتریس داده اصلی و بازسازی آن به عنوان خطای بازسازی محاسبه می‌شود. داده‌های پرت بر اساس خطاهای بازسازی غیرعادی بالا شناسایی می‌گردند.
3. **تقریب با رتبه پایین:** تجزیه ماتریس با هدف تقریب ماتریس داده اصلی با یک تقریب با رتبه پایین (low-rank approximation)، ثبت ویژگی‌های ضروری و در عین حال کاهش تأثیر نویز و داده‌های پرت انجام می‌شود.
4. **مقاوم در برابر ناهنجاری‌ها:** روش‌های تجزیه ماتریس اغلب در برابر داده‌های پرت مقاوم هستند، زیرا آن‌ها بر ثبت الگوهای غالب در داده‌ها تمرکز می‌کنند.
5. **کاربردها:** این روش‌ها معمولاً برای تشخیص داده پرت در داده‌های عددی، مانند قرائت‌های حسگر، داده‌های تصویری، و سایر مجموعه‌های داده چندبعدی استفاده می‌شوند.

اکنون، بیا یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تشخیص داده پرت با استفاده از تجزیه ماتریس با تجزیه مقدار منفرد (SVD) ارائه دهیم:

In []: *و خطای بازسازی SVD آموزش: تشخیص نقاط پرت با استفاده از تجزیه ماتریسی #*

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import StandardScaler
```

تولید داده‌های مصنوعی شامل داده‌های نرمال و نقاط پرت **1**

```
np.random.seed(42)
data_normal = np.random.normal(loc=0, scale=1, size=(100, 2))    # داده‌های نرمال
data_outliers = np.random.normal(loc=10, scale=1, size=(5, 2))    # نقاط پرت شدید

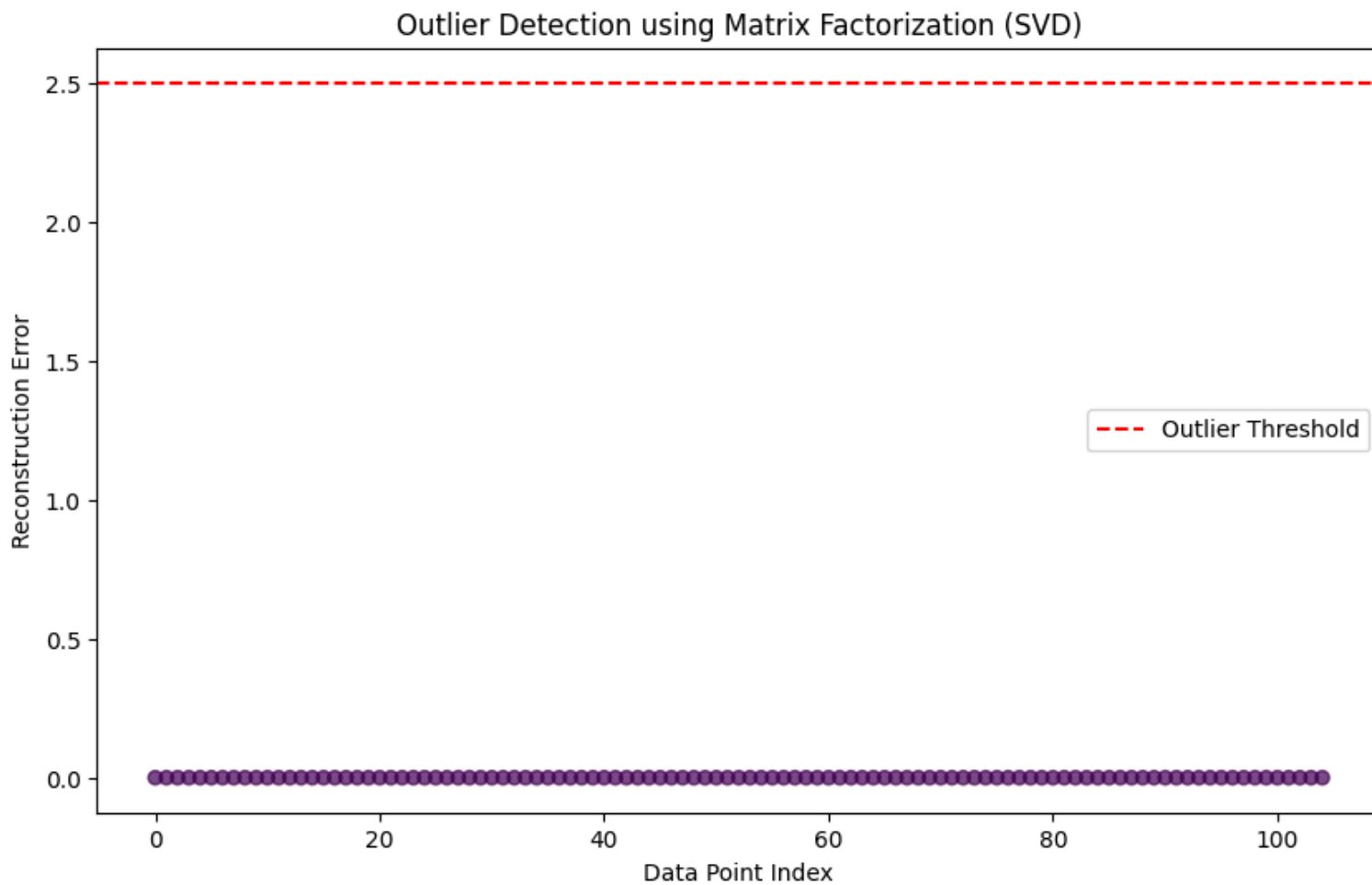
# 2 ترکیب داده‌ها و استانداردسازی ویژگی‌ها
data = np.vstack([data_normal, data_outliers])
scaler = StandardScaler()
data_standardized = scaler.fit_transform(data)

# 3 برای کاهش بعد و بازسازی داده‌ها SVD اعمال
svd = TruncatedSVD(n_components=2)
svd.fit(data_standardized)
reconstructed_data = svd.inverse_transform(svd.transform(data_standardized))

# 4 محاسبه خطای بازسازی برای هر نمونه
reconstruction_error = np.mean(np.square(data_standardized - reconstructed_data), axis=1)

# 5 تعیین آستانه برای تشخیص نقاط پرت
threshold = 2.5
outliers = reconstruction_error > threshold

# 6 نمایش تصویری خطای بازسازی و نقاط پرت
plt.figure(figsize=(10, 6))
plt.scatter(range(len(data)), reconstruction_error, c=outliers, cmap='viridis', alpha=0.7)
plt.axhline(y=threshold, color='r', linestyle='--', label='Outlier Threshold')
plt.title('Outlier Detection using Matrix Factorization (SVD)')
plt.xlabel('Data Point Index')
plt.ylabel('Reconstruction Error')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از روش **تجزیه مقدار منفرد (SVD)** برای تشخیص نقاط پرت در داده‌های دوبعدی استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های نرمال و نقاط پرت مصنوعی تولید شده‌اند و با هم ترکیب شده‌اند.
- داده‌ها با استفاده از `StandardScaler` استانداردسازی شده‌اند تا میانگین صفر و واریانس یک داشته باشند.
- با استفاده از `TruncatedSVD` داده‌ها به فضای فشرده‌تر نگاشته شده و سپس بازسازی شده‌اند.
- برای هر نمونه، خطای بازسازی محاسبه شده و نمونه‌هایی با خطای بالا به‌عنوان نقاط پرت شناسایی شده‌اند.
- نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا نقاط پرت به‌صورت رنگی قابل تشخیص باشند.

روش‌های مبتنی بر بازسازی ماتریسی مانند SVD برای تشخیص ناهنجاری در داده‌های عددی و کاهش بعد بسیار مؤثر هستند. این روش‌ها به‌ویژه در مسائل تشخیص ناهنجاری در داده‌های سنسوری، مالی و تصویری کاربرد دارند.

روش‌های فشرده‌سازی مبتنی بر الگو (Pattern-based compression methods) برای تشخیص داده پرت

روش‌های فشرده‌سازی مبتنی بر الگو برای داده‌های دسته‌بندی‌شده (categorical data) شامل نمایش داده‌های دسته‌بندی‌شده اصلی با استفاده از الگوها یا کدهای فشرده هستند. سپس داده‌های پرت بر اساس عدم تشابه بین الگوهای دسته‌بندی‌شده مشاهده شده و بازسازی شده شناسایی می‌شوند.

نکات کلیدی درباره روش‌های فشرده‌سازی مبتنی بر الگو:

- نمایش داده دسته‌بندی‌شده:** داده‌های دسته‌بندی‌شده با استفاده از الگوها یا کدهای فشرده نمایش داده می‌شوند، که معمولاً از طریق تکنیک‌هایی مانند کاوش الگوهای مکرر (frequent pattern mining) یا طرح‌های کدگذاری به دست می‌آیند.
- خطای بازسازی:** تفاوت بین داده‌های دسته‌بندی‌شده اصلی و شکل بازسازی‌شده آن به عنوان خطای بازسازی محاسبه می‌شود. داده‌های پرت بر اساس خطاهای بازسازی غیرعادی بالا شناسایی می‌گردند.
- تکنیک‌های کاوش الگو:** الگوریتم‌های کاوش الگوهای مکرر، مانند Apriori یا FP-growth، می‌توانند برای کشف الگوهای مهم در داده‌های دسته‌بندی‌شده به کار گرفته شوند.
- کدگذاری یک-داغ:** کدگذاری یک-داغ (One-Hot Encoding) یک طرح کدگذاری رایج است که در آن هر دسته به عنوان یک بردار دودویی نمایش داده می‌شود و فشرده‌سازی با ثبت حضور یا عدم حضور دسته‌ها حاصل می‌شود.
- کاربردها:** این روش‌ها به ویژه برای تشخیص داده پرت در مجموعه‌های داده‌ای که در آن‌ها متغیرهای دسته‌بندی‌شده غالب هستند، مانند داده‌های تراکنش مشتری یا تاریخچه خرید اقلام، مفید می‌باشند.

اکنون، بیا یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تشخیص داده پرت با استفاده از روش‌های فشرده‌سازی مبتنی بر الگو با کدگذاری یک-داغ ارائه دهیم:

In []: و خطای بازسازی SVD آموزش: تشخیص نقاط پرت در داده‌های دسته‌ای با استفاده از #

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
from sklearn.decomposition import TruncatedSVD

# 1 تولید داده‌های دسته‌ای مصنوعی شامل مقادیر پرت
data = pd.DataFrame({
    'Category_A': ['A', 'B', 'A', 'C', 'B', 'A', 'A'],
    'Category_B': ['X', 'Y', 'X', 'Z', 'Y', 'Z', 'X'],
    'Category_C': ['High', 'Low', 'Medium', 'High', 'Medium', 'Low', 'Low']
})

# 2 One-Hot Encoding تبدیل ویژگی‌های دسته‌ای به بردارهای دودویی با
encoder = OneHotEncoder()
encoded_data = encoder.fit_transform(data).toarray()

# 3 برای کاهش بعد و بازسازی داده‌ها SVD اعمال
svd = TruncatedSVD(n_components=2)
svd.fit(encoded_data)
reconstructed_data = svd.inverse_transform(svd.transform(encoded_data))

# 4 محاسبه خطای بازسازی برای هر نمونه
reconstruction_error = ((encoded_data - reconstructed_data) ** 2).sum(axis=1)

# 5 تعیین آستانه برای تشخیص نقاط پرت
threshold = 0.1
outliers = reconstruction_error > threshold

# 6 افزودن نتایج به دیتافریم اصلی و نمایش
data['Reconstruction Error'] = reconstruction_error
data['Outlier'] = outliers.astype(int)
print(data)
```

	Category_A	Category_B	Category_C	Reconstruction Error	Outlier
0	A	X	High	0.746753	1
1	B	Y	Low	0.481770	1
2	A	X	Medium	1.131273	1
3	C	Z	High	2.623078	1
4	B	Y	Medium	0.617257	1
5	A	Z	Low	1.426947	1
6	A	X	Low	0.566305	1

توضیحات مثال کد پایتون

در این مثال، از روش تجزیه مقدار منفرد (SVD) برای تشخیص نقاط پرت در داده‌های دسته‌ای استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های دسته‌ای مصنوعی شامل سه ویژگی متنی تولید شده‌اند.
- با استفاده از OneHotEncoder داده‌ها به بردارهای دودویی تبدیل شده‌اند تا قابل پردازش عددی باشند.
- با استفاده از TruncatedSVD داده‌ها به فضای فشرده‌تر نگاشته شده و سپس بازسازی شده‌اند.
- برای هر نمونه، خطای بازسازی محاسبه شده و نمونه‌هایی با خطای بالا به‌عنوان نقاط پرت شناسایی شده‌اند.
- نتایج نهایی شامل خطای بازسازی و برچسب پرت بودن به دیتافریم اصلی اضافه شده‌اند.

این روش برای تشخیص ناهنجاری در داده‌های دسته‌ای بسیار مفید است، به‌ویژه زمانی که داده‌ها به‌صورت بردارهای sparse نمایش داده می‌شوند. استفاده از SVD به کاهش بعد و استخراج ساختارهای پنهان کمک می‌کند.

Section 11.5

11.5.1. Clustering-based approaches

رویکردهای مبتنی بر خوشه‌بندی (Clustering-based approaches) برای تشخیص داده پرت

رویکردهای مبتنی بر خوشه‌بندی برای تشخیص داده پرت شامل گروه‌بندی نقاط داده به خوشه‌ها بر اساس شباهت و سپس شناسایی داده‌های پرت به عنوان نقاطی است که به خوبی با هیچ خوشه‌ای مطابقت ندارند. داده‌های پرت اغلب به عنوان نقاط داده‌ای در نظر گرفته می‌شوند که در خوشه‌های تشکیل شده قرار نمی‌گیرند.

نکات کلیدی درباره رویکردهای مبتنی بر خوشه‌بندی:

- تشکیل خوشه:** نقاط داده بر اساس شباهت یا نزدیکی به خوشه‌ها گروه‌بندی می‌شوند. الگوریتم‌های خوشه‌بندی رایج شامل K-Means، خوشه‌بندی سلسله‌مراتبی (hierarchical clustering) و DBSCAN هستند.
- شناسایی داده پرت:** داده‌های پرت، نقاطی هستند که به هیچ خوشه مشخصی تعلق ندارند یا به خوشه‌های کوچک یا نویز (noise) اختصاص داده می‌شوند. این نقاط از مرکزهای (centroids) خوشه‌های تشکیل شده دور هستند.
- معیارهای فاصله:** معیارهای فاصله، مانند فاصله اقلیدسی (Euclidean distance) یا فاصله مالهالانوبیس (Mahalanobis distance)، اغلب برای ارزیابی شباهت بین نقاط داده و مراکز خوشه استفاده می‌شوند.
- اندازه و چگالی خوشه:** داده‌های پرت اغلب در خوشه‌هایی با اندازه‌های کوچک یا چگالی کم شناسایی می‌شوند، زیرا این خوشه‌ها ممکن است نشان‌دهنده ناهنجاری‌هایی (anomalies) در داده‌ها باشند.

5. **کاربردها:** تشخیص داده پرت مبتنی بر خوشه‌بندی در طیف وسیعی از مجموعه‌های داده، از جمله آن‌هایی که دارای ویژگی‌های عددی و دسته‌بندی‌شده هستند، قابل اجرا است.

اکنون، بیاید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تشخیص داده پرت با استفاده از الگوریتم خوشه‌بندی DBSCAN ارائه دهیم:

```
In [ ]: # (مبتنی بر خوشه‌بندی) DBSCAN آموزش: تشخیص نقاط پرت با استفاده از الگوریتم

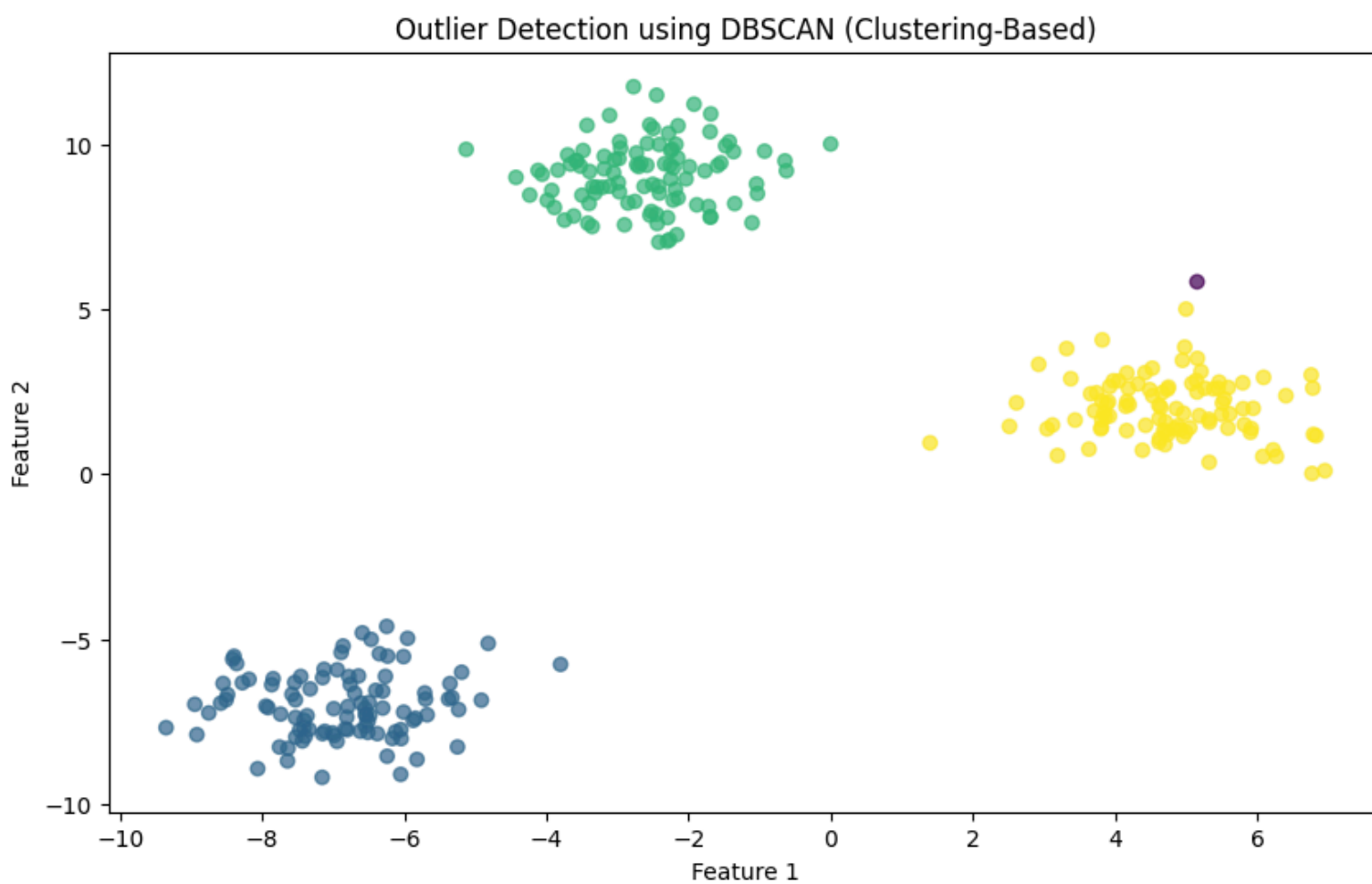
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs

# 1 تولید داده‌های مصنوعی با سه خوشه و دو نقطه پرت
data, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)
outliers = np.array([[0, 10], [5, 5]]) # نقاط پرت خارج از خوشه‌ها

# 2 ترکیب داده‌های نرمال با نقاط پرت
data = np.vstack([data, outliers])

# 3 برای خوشه‌بندی و تشخیص نقاط پرت DBSCAN اعمال الگوریتم
dbscan = DBSCAN(eps=1.5, min_samples=5)
labels = dbscan.fit_predict(data) # برچسب‌های 1- (پرت) و عددهای خوشه

# 4 نمایش تصویری نتایج خوشه‌بندی و نقاط پرت
plt.figure(figsize=(10, 6))
plt.scatter(data[:, 0], data[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.title('Outlier Detection using DBSCAN (Clustering-Based)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از الگوریتم **DBSCAN** برای تشخیص نقاط پرت و خوشه‌بندی داده‌های دوبعدی استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با سه خوشه مشخص تولید شده‌اند و دو نقطه پرت به‌صورت دستی اضافه شده‌اند.
- الگوریتم DBSCAN با پارامترهای `eps=1.5` و `min_samples=5` اجرا شده و نقاط پرت را با برچسب -1 مشخص کرده است.
- نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا خوشه‌ها و نقاط پرت به‌صورت رنگی قابل تشخیص باشند.

DBSCAN یکی از الگوریتم‌های قدرتمند برای خوشه‌بندی داده‌هایی با ساختار پیچیده و تشخیص ناهنجاری است. این روش بدون نیاز به تعیین تعداد خوشه‌ها، بر اساس چگالی محلی عمل می‌کند و نقاط پرت را به‌صورت طبیعی جدا می‌سازد.

11.5.2. Classification-based approaches

رویکردهای مبتنی بر طبقه‌بندی (Classification-based approaches) برای تشخیص داده پرت

رویکردهای مبتنی بر طبقه‌بندی برای تشخیص داده پرت شامل آموزش یک مدل یادگیری ماشین برای تمایز بین نمونه‌های عادی و ناهنجار (anomalous instances) است. مدل بر روی یک مجموعه داده دارای برچسب (labeled dataset) آموزش داده می‌شود، جایی که نمونه‌های عادی کلاس اکثریت و داده‌های پرت کلاس اقلیت هستند. پس از آموزش، مدل می‌تواند پیش‌بینی کند که آیا یک نمونه جدید عادی است یا یک داده پرت.

نکات کلیدی درباره رویکردهای مبتنی بر طبقه‌بندی:

- یادگیری با نظارت:** رویکردهای مبتنی بر طبقه‌بندی روش‌های یادگیری با نظارت هستند که برای آموزش به داده‌های دارای برچسب نیاز دارند. مجموعه داده باید شامل نمونه‌هایی باشد که به عنوان عادی یا داده پرت برچسب‌گذاری شده‌اند.
- انتخاب مدل:** الگوریتم‌های طبقه‌بندی رایج مانند ماشین‌های بردار پشتیبان (Support Vector Machines - SVM)، جنگل‌های تصادفی (Random Forests) یا شبکه‌های عصبی (Neural Networks) می‌توانند برای ساخت مدل‌های تشخیص داده پرت به کار گرفته شوند.
- مدیریت داده نامتعادل:** مجموعه‌های داده تشخیص داده پرت اغلب نامتعادل هستند، به طوری که اکثریت نمونه‌ها عادی می‌باشند. مدیریت عدم تعادل کلاس برای عملکرد مدل حیاتی است و تکنیک‌هایی مانند بیش‌نمونه‌گیری (oversampling)، کم‌نمونه‌گیری (undersampling) یا استفاده از الگوریتم‌های تخصصی به کار می‌روند.
- آستانه تصمیم:** تصمیم مدل بر اساس یک آستانه احتمال یا امتیاز (score) است. نمونه‌هایی با احتمال زیر آستانه، به عنوان داده پرت طبقه‌بندی می‌شوند.
- کاربردها:** تشخیص داده پرت مبتنی بر طبقه‌بندی متنوع بوده و برای حوزه‌های مختلفی از جمله تشخیص تقلب، امنیت شبکه و مراقبت‌های بهداشتی قابل استفاده است.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تشخیص داده پرت با استفاده از یک طبقه‌بند ماشین بردار پشتیبان (SVM) ارائه دهیم:

In []: (مبتنی بر طبقه‌بندی) One-Class SVM آموزش: تشخیص نقاط پرت با استفاده از الگوریتم

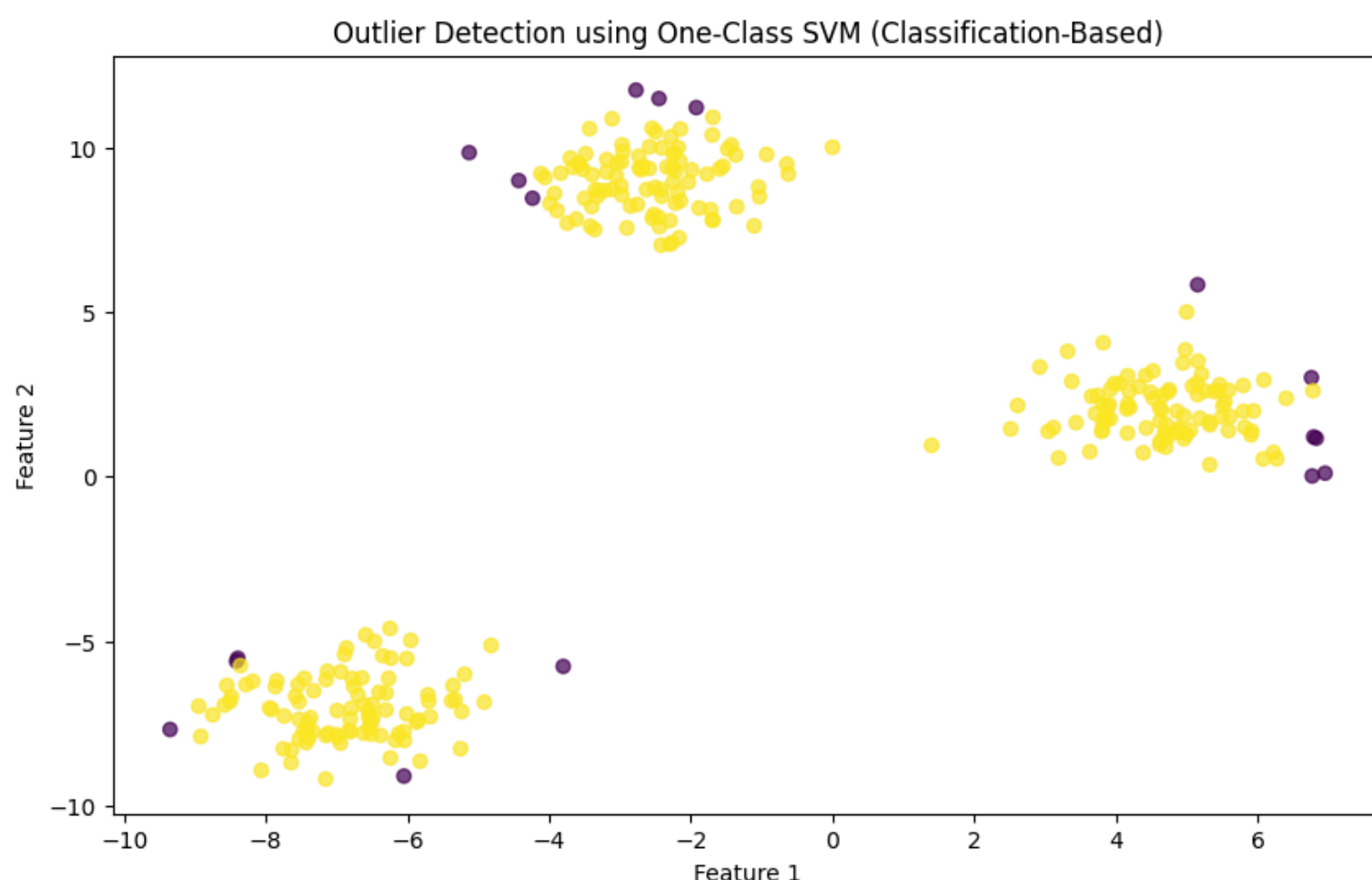
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import OneClassSVM
from sklearn.datasets import make_blobs

# تولید داده‌های مصنوعی با سه خوشه و دو نقطه پرت
data, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)
outliers = np.array([[0, 10], [5, 5]]) # نقاط پرت خارج از خوشه‌ها

# ترکیب داده‌های نرمال با نقاط پرت
data = np.vstack([data, outliers])

# برای تشخیص نقاط پرت One-Class SVM اعمال الگوریتم
svm = OneClassSVM(nu=0.05) # نسبت تقریبی نقاط پرت
```

```
labels = svm.fit_predict(data) # (پرت) 1 و (نرمال) 0
# نمایش تصویری نتایج تشخیص
plt.figure(figsize=(10, 6))
plt.scatter(data[:, 0], data[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.title('Outlier Detection using One-Class SVM (Classification-Based)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از الگوریتم **One-Class SVM** برای تشخیص نقاط پرت در داده‌های دوبعدی استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با سه خوشه مشخص تولید شده‌اند و دو نقطه پرت به‌صورت دستی اضافه شده‌اند.
- الگوریتم One-Class SVM با پارامتر `nu=0.05` اجرا شده که بیانگر نسبت تقریبی نقاط پرت در داده‌هاست.
- مدل با استفاده از داده‌های نرمال آموزش دیده و نقاط پرت را با برچسب 1- شناسایی کرده است.
- نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا نقاط پرت و نرمال به‌صورت رنگی قابل تشخیص باشند.

الگوریتم One-Class SVM برای تشخیص ناهنجاری در داده‌هایی که فقط نمونه‌های نرمال در دسترس هستند بسیار مناسب است. این روش بر اساس مرز تصمیم‌گیری در فضای ویژگی‌ها عمل می‌کند و نقاط خارج از آن را پرت در نظر می‌گیرد.

Section 11.6

11.6.1. Transforming contextual outlier detection to conventional outlier detection

تبدیل تشخیص داده پرت زمینه‌ای به تشخیص داده پرت مرسوم

تشخیص داده پرت زمینه‌ای (Contextual outlier detection) شامل در نظر گرفتن زمینه یا محیطی است که نقاط داده در آن وجود دارند. تبدیل تشخیص داده پرت زمینه‌ای به تشخیص داده پرت مرسوم اغلب شامل استخراج ویژگی‌ها (features) یا

نمایش‌های مرتبط از اطلاعات زمینه‌ای است که امکان استفاده از تکنیک‌های استاندارد تشخیص داده پرت را فراهم می‌سازد.

نکات کلیدی درباره تبدیل تشخیص داده پرت زمینه‌ای:

1. **اطلاعات زمینه‌ای:** تشخیص داده پرت زمینه‌ای، اطلاعات یا ویژگی‌های اضافی را که زمینه یا محیط نقاط داده را توصیف

می‌کنند، در نظر می‌گیرد. این می‌تواند شامل اطلاعات زمانی، اطلاعات مکانی، یا هر زمینه مرتبط دیگری باشد.

2. **استخراج ویژگی:** برای تبدیل تشخیص داده پرت زمینه‌ای به تشخیص داده پرت مرسوم، اغلب تکنیک‌های استخراج

ویژگی برای تقطیر جنبه‌های مهم اطلاعات زمینه‌ای به ویژگی‌های عددی اعمال می‌شوند.

3. **تکنیک‌های مرسوم:** هنگامی که ویژگی‌های مرتبط استخراج شدند، تکنیک‌های مرسوم تشخیص داده پرت، مانند

روش‌های مبتنی بر خوشه‌بندی (clustering) یا طبقه‌بندی (classification)، می‌توانند برای شناسایی داده‌های پرت به

کار روند.

4. **کاربردها:** این رویکرد زمانی مفید است که با داده‌هایی سروکار داریم که اطلاعات زمینه‌ای در آن‌ها موجود است، اما

باید به قالبی مناسب برای روش‌های استاندارد تشخیص داده پرت ترجمه شود.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تبدیل تشخیص داده پرت زمینه‌ای با استفاده از

استخراج ویژگی و اعمال یک رویکرد مبتنی بر خوشه‌بندی ارائه دهیم:

In []: *# PCA و DBSCAN آموزش: تشخیص نقاط پرت زمینه‌ای با استفاده از*

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA

# تولید داده‌های مصنوعی شامل داده‌های نرمال و نقاط پرت 1
np.random.seed(42)
data_contextual = np.random.normal(loc=0, scale=1, size=(100, 2)) # داده‌های نرمال
data_outliers = np.random.normal(loc=10, scale=1, size=(5, 2)) # نقاط پرت زمینه‌ای

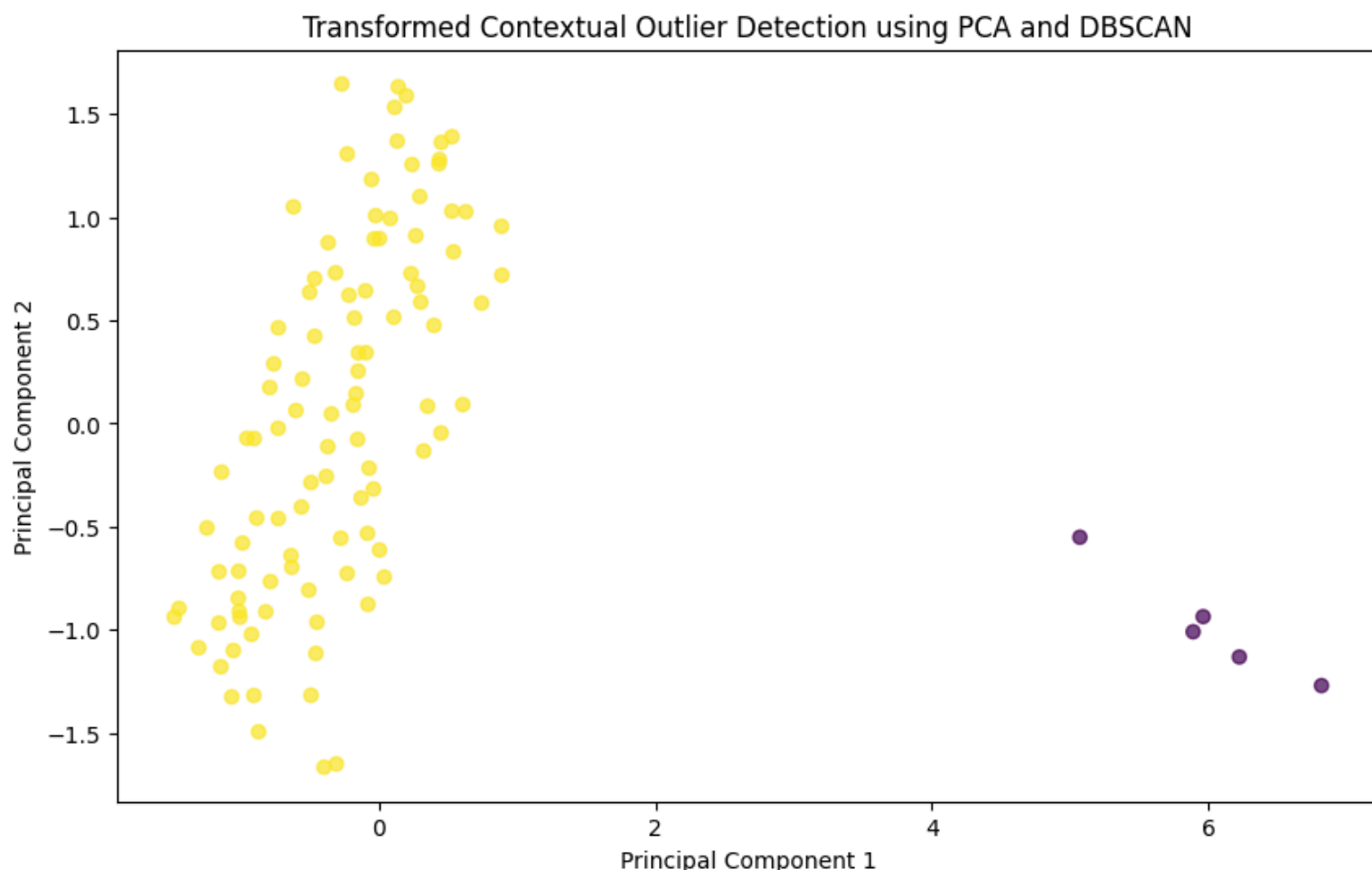
# ترکیب داده‌ها و افزودن اطلاعات زمینه‌ای (زمانی) 2
data = np.vstack([data_contextual, data_outliers])
temporal_context = np.arange(0, 105).reshape(-1, 1) # اطلاعات زمانی به‌عنوان زمینه
data_with_context = np.hstack([data, temporal_context]) # ترکیب ویژگی‌ها با زمینه

# استانداردسازی داده‌ها برای حذف اثر مقیاس 3
scaler = StandardScaler()
data_standardized = scaler.fit_transform(data_with_context)

# استخراج ویژگی‌های اصلی با استفاده از PCA 4
pca = PCA(n_components=2)
data_pca = pca.fit_transform(data_standardized)

# برای تشخیص نقاط پرت بر اساس ویژگی‌های استخراج‌شده DBSCAN اعمال 5
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(data_pca) # برچسب‌های 1- (پرت) و عددهای خوشه

# نمایش تصویری نتایج تشخیص 6
plt.figure(figsize=(10, 6))
plt.scatter(data_pca[:, 0], data_pca[:, 1], c=labels, cmap='viridis', alpha=0.7)
plt.title('Transformed Contextual Outlier Detection using PCA and DBSCAN')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از ترکیب تحلیل مؤلفه‌های اصلی (PCA) و DBSCAN برای تشخیص نقاط پرت زمینه‌ای استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های نرمال و نقاط پرت مصنوعی تولید شده‌اند و با هم ترکیب شده‌اند.
- اطلاعات زمینه‌ای (مانند زمان یا ترتیب وقوع) به‌عنوان ویژگی سوم به داده‌ها اضافه شده است.
- داده‌ها با استفاده از `StandardScaler` استانداردسازی شده‌اند تا اثر مقیاس حذف شود.
- با استفاده از `PCA` داده‌ها به فضای دوبعدی فشرده شده‌اند تا ویژگی‌های اصلی استخراج شوند.
- الگوریتم **DBSCAN** روی فضای PCA اجرا شده و نقاط پرت را با برچسب 1- شناسایی کرده است.
- نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا خوشه‌ها و نقاط پرت قابل مشاهده باشند.

این روش برای تشخیص ناهنجاری‌های زمینه‌ای (contextual outliers) بسیار مؤثر است، به‌ویژه زمانی که اطلاعات اضافی مانند زمان، مکان یا شرایط محیطی در داده‌ها وجود دارد. ترکیب کاهش بعد و خوشه‌بندی می‌تواند ساختارهای پنهان را آشکار کند.

11.6.2. Modeling normal behavior with respect to contexts

مدل‌سازی رفتار عادی با توجه به زمینه‌ها (Modeling normal behavior with respect to contexts)

مدل‌سازی رفتار عادی با توجه به زمینه‌ها شامل ساخت مدل‌هایی است که الگوها یا رفتارهای معمول را در زمینه‌های خاص ثبت می‌کنند. این رویکرد با هدف تمایز بین نمونه‌های عادی و ناهنجار (anomalous instances) بر اساس مدل‌های رفتار عادی که مختص زمینه یاد گرفته شده‌اند، صورت می‌گیرد.

نکات کلیدی درباره مدل‌سازی رفتار عادی با توجه به زمینه‌ها:

- **اطلاعات زمینه‌ای:** در نظر گرفتن اطلاعات زمینه‌ای ضروری است. این می‌تواند شامل عوامل محیطی، الگوهای مبتنی بر زمان، یا هر ویژگی (attribute) دیگری که مختص زمینه است، باشد.
- **مدل‌سازی رفتار عادی:** رفتار عادی در هر زمینه مدل‌سازی می‌شود تا الگوها یا توزیع‌های مورد انتظار درک شوند. این مدل‌سازی اغلب با استفاده از روش‌های آماری، مدل‌های یادگیری ماشین، یا دانش خاص حوزه انجام می‌شود.
- **تشخیص ناهنجاری:** ناهنجاری‌ها با مقایسه نمونه‌های مشاهده‌شده با رفتار عادی مدل‌سازی‌شده شناسایی می‌شوند. نمونه‌هایی که به طور قابل توجهی از الگوهای مورد انتظار منحرف می‌شوند، داده پرت (outliers) در نظر گرفته می‌شوند.
- **زمینه‌های پویا:** در زمینه‌های پویا، ممکن است مدل‌ها نیاز به انطباق در طول زمان داشته باشند؛ زیرا رفتار عادی تکامل می‌یابد. نظارت مداوم و آموزش مجدد مدل‌ها ممکن است ضروری باشد.
- **کاربردها:** این رویکرد برای سناریوهایی مناسب است که رفتار عادی در زمینه‌های مختلف متفاوت است و برای تشخیص مؤثر داده پرت، درک آگاهانه از زمینه مورد نیاز است.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای مدل‌سازی رفتار عادی با توجه به زمینه‌ها با استفاده از یک مدل ترکیبی گوسی (Gaussian Mixture Model - GMM) ارائه دهیم:

In []: و تشخیص نقاط پرت زمینه‌ای GMM آموزش: مدل‌سازی رفتار نرمال با استفاده از #

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

# تولید داده‌های مصنوعی شامل داده‌های نرمال و نقاط پرت زمینه‌ای 1
np.random.seed(42)
data_contextual = np.random.normal(loc=0, scale=1, size=(100, 2)) # داده‌های نرمال
data_outliers = np.random.normal(loc=10, scale=1, size=(5, 2)) # نقاط پرت زمینه‌ای

# ترکیب داده‌ها و افزودن اطلاعات زمینه‌ای (مانند زمان یا ترتیب وقوع) 2
data = np.vstack([data_contextual, data_outliers])
temporal_context = np.arange(0, 105).reshape(-1, 1) # ویژگی زمینه‌ای
data_with_context = np.hstack([data, temporal_context]) # ترکیب ویژگی‌ها با زمینه

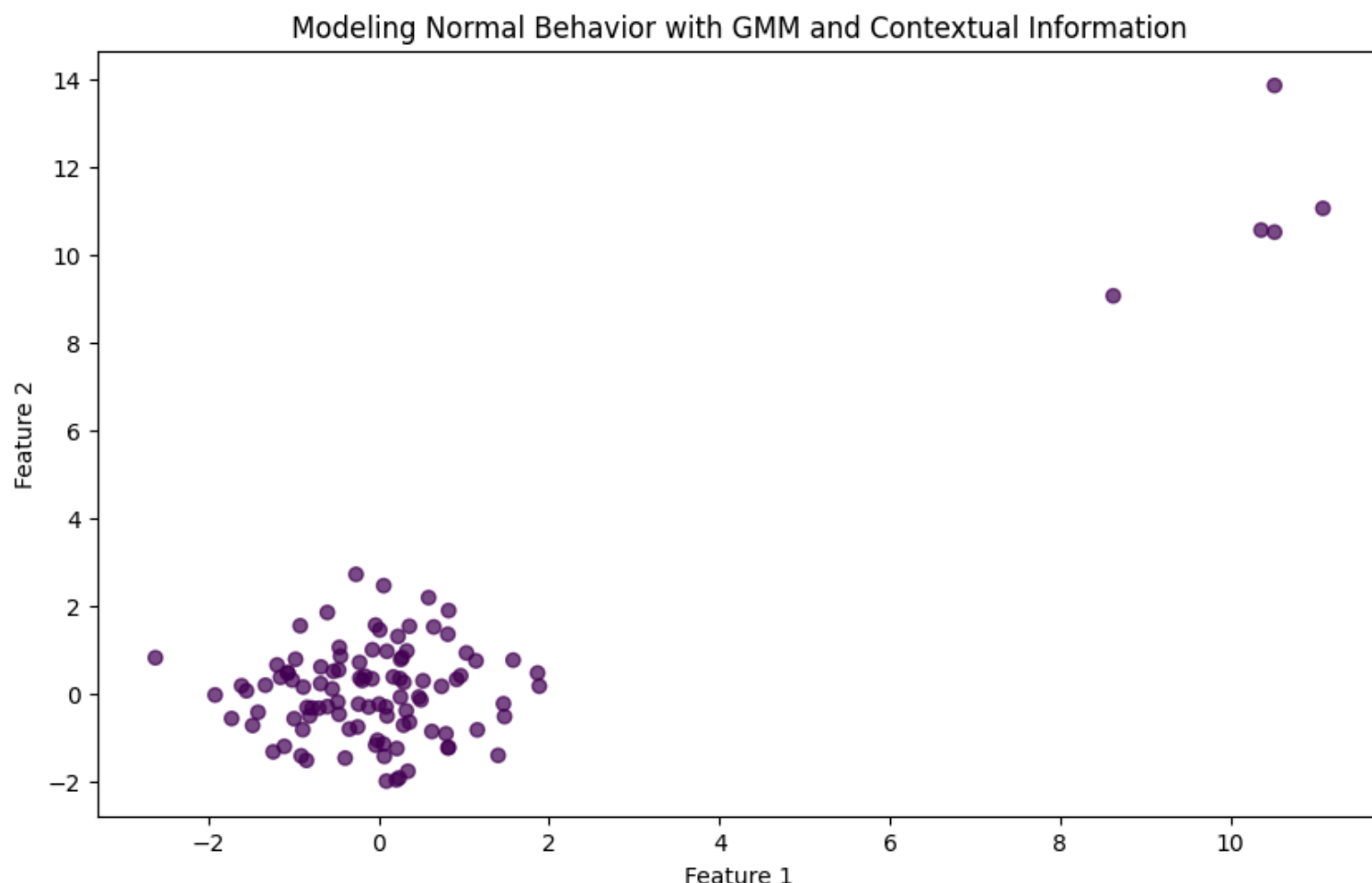
# استانداردسازی داده‌ها برای حذف اثر مقیاس 3
scaler = StandardScaler()
data_standardized = scaler.fit_transform(data_with_context)

# برای مدل‌سازی رفتار نرمال GMM اعمال مدل 4
gmm = GaussianMixture(n_components=2, covariance_type='full', random_state=42)
gmm.fit(data_standardized)

# برای هر نمونه log-likelihood محاسبه 5
probabilities = gmm.score_samples(data_standardized)

# تعیین آستانه برای تشخیص نقاط پرت 6
threshold = -8
outliers = probabilities < threshold # نمونه‌هایی با احتمال پایین‌تر از آستانه

# نمایش تصویری نتایج تشخیص 7
plt.figure(figsize=(10, 6))
plt.scatter(data_with_context[:, 0], data_with_context[:, 1], c=outliers, cmap='viridis', alpha=0.7)
plt.title('Modeling Normal Behavior with GMM and Contextual Information')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از مدل **Gaussian Mixture Model (GMM)** برای مدل‌سازی رفتار نرمال و تشخیص نقاط پرت زمینه‌ای استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های نرمال و نقاط پرت مصنوعی تولید شده‌اند و با هم ترکیب شده‌اند.
- اطلاعات زمینه‌ای (مانند زمان یا ترتیب وقوع) به‌عنوان ویژگی سوم به داده‌ها اضافه شده است.
- داده‌ها با استفاده از `StandardScaler` استانداردسازی شده‌اند تا اثر مقیاس حذف شود.
- مدل **GMM** با دو مؤلفه اجرا شده و برای هر نمونه مقدار $\log\text{-likelihood}$ محاسبه شده است.
- نمونه‌هایی با احتمال پایین‌تر از آستانه تعیین‌شده به‌عنوان نقاط پرت شناسایی شده‌اند.
- نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا نقاط پرت قابل مشاهده باشند.

مدل‌های آماری مانند GMM برای تشخیص ناهنجاری در داده‌هایی با ساختار پیچیده و زمینه‌ای بسیار مؤثر هستند. این روش‌ها می‌توانند رفتار نرمال را مدل‌سازی کرده و انحراف از آن را به‌صورت کمی شناسایی کنند.

11.6.3. Mining collective outliers

کاوش داده‌های پرت جمعی (Mining collective outliers)

کاوش داده‌های پرت جمعی شامل شناسایی گروه‌ها یا زیرمجموعه‌هایی از نمونه‌ها است که به طور جمعی رفتار ناهنجار از خود نشان می‌دهند. به جای تمرکز بر روی نقاط داده فردی، این رویکرد به دنبال الگوهای ناهنجاری می‌گردد که با در نظر گرفتن تعاملات یا روابط بین چندین نمونه ظهور می‌کنند.

نکات کلیدی درباره کاوش داده‌های پرت جمعی:

- **ناهنجاری‌های جمعی:** داده‌های پرت جمعی ناهنجاری‌هایی هستند که هنگام در نظر گرفتن روابط، تعاملات یا

وابستگی‌ها در میان چندین نمونه به جای نمونه‌های فردی، پدیدار می‌شوند.

- **مدل‌سازی رابطه:** کاوش داده‌های پرت جمعی اغلب شامل مدل‌سازی روابط یا وابستگی‌ها بین نقاط داده است، مانند

قوانین انجمنی (association rules)، رویکردهای مبتنی بر گراف یا سایر اشکال مدل‌سازی رابطه‌ای.

- **الگوهای نوظهور:** داده‌های پرت جمعی ممکن است الگوهای نوظهوری از خود نشان دهند که هنگام تحلیل نمونه‌های

فردی به صورت ایزوله آشکار نیستند. رفتار جمعی به دلیل اثرات ترکیبی گروه، غیرعادی می‌شود.

- **کاربردها:** این رویکرد برای سناریوهایی مفید است که در آن‌ها ناهنجاری‌ها در بستر روابط یا تعاملات بهتر درک

می‌شوند، مانند تشخیص تقلب در شبکه‌ها، تحلیل رفتار گروهی یا فیلترگذاری مشارکتی (collaborative filtering).

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای کاوش داده‌های پرت جمعی با استفاده از یک

رویکرد مبتنی بر گراف ارائه دهیم:

In []: آموزش: تشخیص نقاط پرت جمعی با استفاده از گراف و نمایش درجه گره‌ها به‌صورت رنگ و اندازه #

```
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
from sklearn.datasets import make_blobs
from matplotlib.colors import Normalize
from matplotlib.cm import ScalarMappable

# 1 تولید داده‌های مصنوعی شامل سه خوشه و دو نقطه پرت
data, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)
outliers = np.array([[0, 10], [5, 5]]) # نقاط پرت خارج از خوشه‌ها
data = np.vstack([data, outliers])

# 2 ساخت گراف بدون وزن بر اساس نزدیکی نقاط
G = nx.Graph()
for i in range(data.shape[0]):
    G.add_node(i)

threshold = 1 # آستانه فاصله برای اتصال پال
for i in range(data.shape[0]):
    for j in range(i + 1, data.shape[0]):
        distance = np.linalg.norm(data[i] - data[j])
        if distance < threshold:
            G.add_edge(i, j)

# 3 محاسبه درجه هر گره (تعداد همسایه‌ها)
degrees = np.array([G.degree[i] for i in range(data.shape[0])])

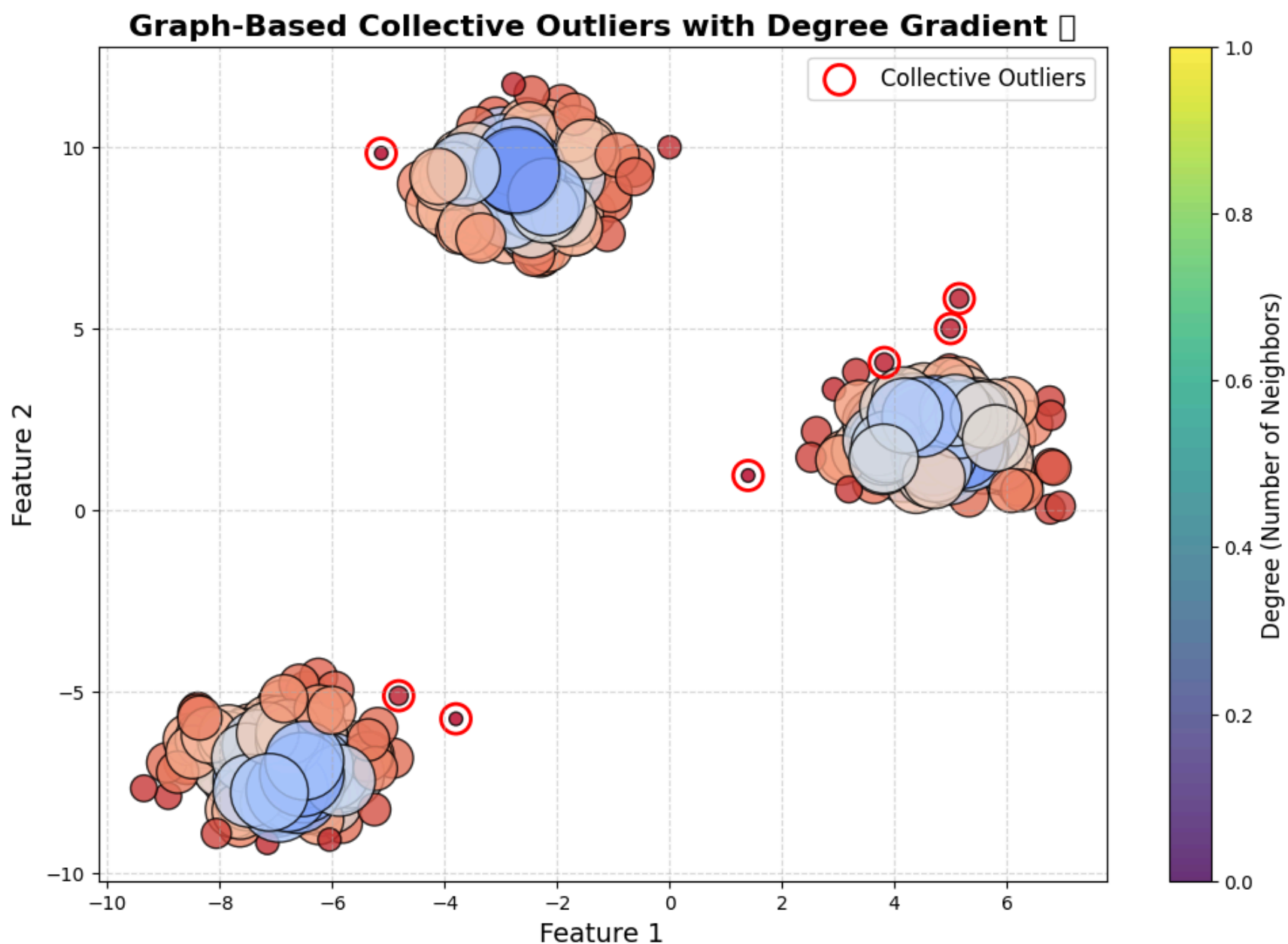
# 4 تنظیم اندازه و رنگ نقاط بر اساس درجه گره
sizes = 50 + 50 * degrees
norm = Normalize(vmin=min(degrees), vmax=max(degrees))
cmap = plt.cm.coolwarm_r
colors = cmap(norm(degrees))

# 5 رسم نقاط با رنگ و اندازه متناسب با درجه
fig, ax = plt.subplots(figsize=(12, 8))
scatter = ax.scatter(data[:, 0], data[:, 1], c=colors, s=sizes, edgecolor='k', alpha=0.8)

# 6 برای نمایش درجه گره‌ها colorbar افزودن
cbar = fig.colorbar(scatter, ax=ax)
cbar.set_label('Degree (Number of Neighbors)', fontsize=12)

# 7 برجسته‌سازی نقاط پرت جمعی (گره‌هایی با درجه کم)
collective_outliers = [i for i in G.nodes if G.degree[i] <= 1]
ax.scatter(data[collective_outliers, 0], data[collective_outliers, 1],
           facecolors='none', edgecolors='red', s=250, linewidths=2, marker='o', label='Collective Outliers')

# 8 Legend تنظیم عنوان، برجسب‌ها و
ax.set_title('Graph-Based Collective Outliers with Degree Gradient 🌈', fontsize=16, fontweight='bold')
ax.set_xlabel('Feature 1', fontsize=14)
ax.set_ylabel('Feature 2', fontsize=14)
ax.grid(True, linestyle='--', alpha=0.5)
ax.legend(fontsize=12)
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از ساختار گراف برای تشخیص **نقاط پرت جمعی** استفاده شده و درجه گره‌ها به صورت رنگ و اندازه نمایش داده شده‌اند. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با سه خوشه تولید شده‌اند و دو نقطه پرت به صورت دستی اضافه شده‌اند.
- بین هر دو نقطه‌ای که فاصله‌شان کمتر از آستانه تعیین‌شده باشد، یال ایجاد شده است.
- درجه هر گره (تعداد همسایه‌ها) محاسبه شده و به صورت رنگی و اندازه‌ای در نمودار نمایش داده شده است.
- گره‌هایی با درجه کمتر یا مساوی ۱ به عنوان نقاط پرت جمعی شناسایی و با حاشیه قرمز برجسته شده‌اند.

این روش برای تحلیل ساختارهای گرافی و شناسایی ناهنجاری‌های گروهی بسیار مؤثر است. ترکیب گراف و نمایش بصری درجه گره‌ها می‌تواند در تحلیل شبکه‌های اجتماعی، داده‌های مکانی و سیستم‌های پیچیده کاربرد فراوانی داشته باشد.

Section 11.7

11.7.1. Extending conventional outlier detection

گسترش روش‌های مرسوم تشخیص داده پرت به داده‌های با ابعاد بالا (High-dimensional data)

گسترش روش‌های مرسوم تشخیص داده پرت به داده‌های با ابعاد بالا (High-dimensional data) شامل پرداختن به چالش‌هایی است که هنگام کار با مجموعه‌های داده با تعداد زیادی ویژگی به وجود می‌آیند. رویکردهای سنتی ممکن است به دلیل پدیده نفرین ابعاد (Curse of Dimensionality) با مشکل مواجه شوند و برای شناسایی مؤثر داده‌های پرت در فضاهای با ابعاد بالا، به اصلاحات یا تکنیک‌های تخصصی نیاز است.

نکات کلیدی درباره گسترش تشخیص داده پرت مرسوم برای داده‌های با ابعاد بالا:

- **نفرین ابعاد (Curse of Dimensionality):** در فضاهای با ابعاد بالا، نقاط داده پراکنده‌تر می‌شوند و فواصل بین نقاط تمایل به افزایش دارند، که باعث می‌شود روش‌های مرسوم مبتنی بر فاصله کمتر مؤثر باشند.
- **کاهش ابعاد (Dimensionality Reduction):** تکنیک‌هایی مانند تحلیل مؤلفه اصلی (Principal Component Analysis - PCA) یا تعبیه‌سازی همسایگی تصادفی توزیع‌شده (t-Distributed Stochastic Neighbor Embedding - t-SNE) می‌توانند برای کاهش ابعاد داده‌ها در عین حفظ اطلاعات مهم به کار گرفته شوند.
- **مدل‌های پراکنده (Sparse Models):** داده‌های با ابعاد بالا اغلب پراکندگی از خود نشان می‌دهند، به طوری که تنها زیرمجموعه کوچکی از ویژگی‌ها ممکن است به داده‌های پرت کمک کنند. مدل‌های پراکنده، مانند LASSO، می‌توانند به شناسایی ویژگی‌های مرتبط کمک کنند.
- **روش‌های ترکیبی (Ensemble Methods):** ترکیب چندین مدل تشخیص داده پرت یا استفاده از روش‌های ترکیبی (Ensemble) که برای داده‌های با ابعاد بالا طراحی شده‌اند، می‌تواند عملکرد کلی را بهبود بخشد.
- **کاربردها (Applications):** این رویکرد برای حوزه‌هایی با مجموعه‌های داده حاوی تعداد زیادی ویژگی، مانند ژنومیک، تحلیل تصویر، یا داده‌های مالی با متغیرهای متعدد، حیاتی است.

اکنون، بیا یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای گسترش تشخیص داده پرت مرسوم به داده‌های با ابعاد بالا با استفاده از روش‌های ترکیبی ارائه دهیم:

In []: *# PCA، Isolation Forest و Sparse Models (LASSO) آموزش: تشخیص نقاط پرت در داده‌های با ابعاد بالا با استفاده از*

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from sklearn.datasets import make_classification, load_iris
from sklearn.decomposition import PCA
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler

# 1 تولید داده‌های مصنوعی با ۲۰ ویژگی (۱۰ اطلاع‌رسان، ۵ افزونه)
data, _ = make_classification(n_samples=500, n_features=20, n_informative=10,
                             n_redundant=5, n_clusters_per_class=1, random_state=42)

# 2 تولید ۱۰ نقطه پرت با پراکندگی بالا
outliers = np.random.normal(loc=0, scale=5, size=(10, 20))

# 3 ترکیب داده‌های نرمال و پرت
data = np.vstack([data, outliers])

# ⚡ برای انتخاب ویژگی‌های مهم (LASSO) استفاده از مدل پراکنده
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)
lasso = LassoCV(cv=5, random_state=42).fit(data_scaled, np.random.randn(data_scaled.shape[0]))
important_features = np.sum(lasso.coef_ != 0)
print(f"LASSO: {important_features} ویژگی‌های انتخاب‌شده توسط")

# 4 برای تصویرسازی PCA کاهش ابعاد با
pca = PCA(n_components=2)
data_pca = pca.fit_transform(data_scaled)

# 5 Isolation Forest تشخیص نقاط پرت با
isolation_forest = IsolationForest(contamination=0.02, random_state=42)
labels = isolation_forest.fit_predict(data_scaled)

# 6 PCA نمایش تصویری داده‌ها در فضای
plt.figure(figsize=(10, 6))
plt.scatter(data_pca[:, 0], data_pca[:, 1], c=labels, cmap='coolwarm', alpha=0.7, s=80, edgecolor='k')
plt.title('High-Dimensional Outlier Detection using PCA + Isolation Forest + LASSO', fontsize=14)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

# 🌸 7 (با ویژگی‌های نویزی اضافه‌شده Iris) مثال کاربردی روی داده واقعی
iris = load_iris()
```

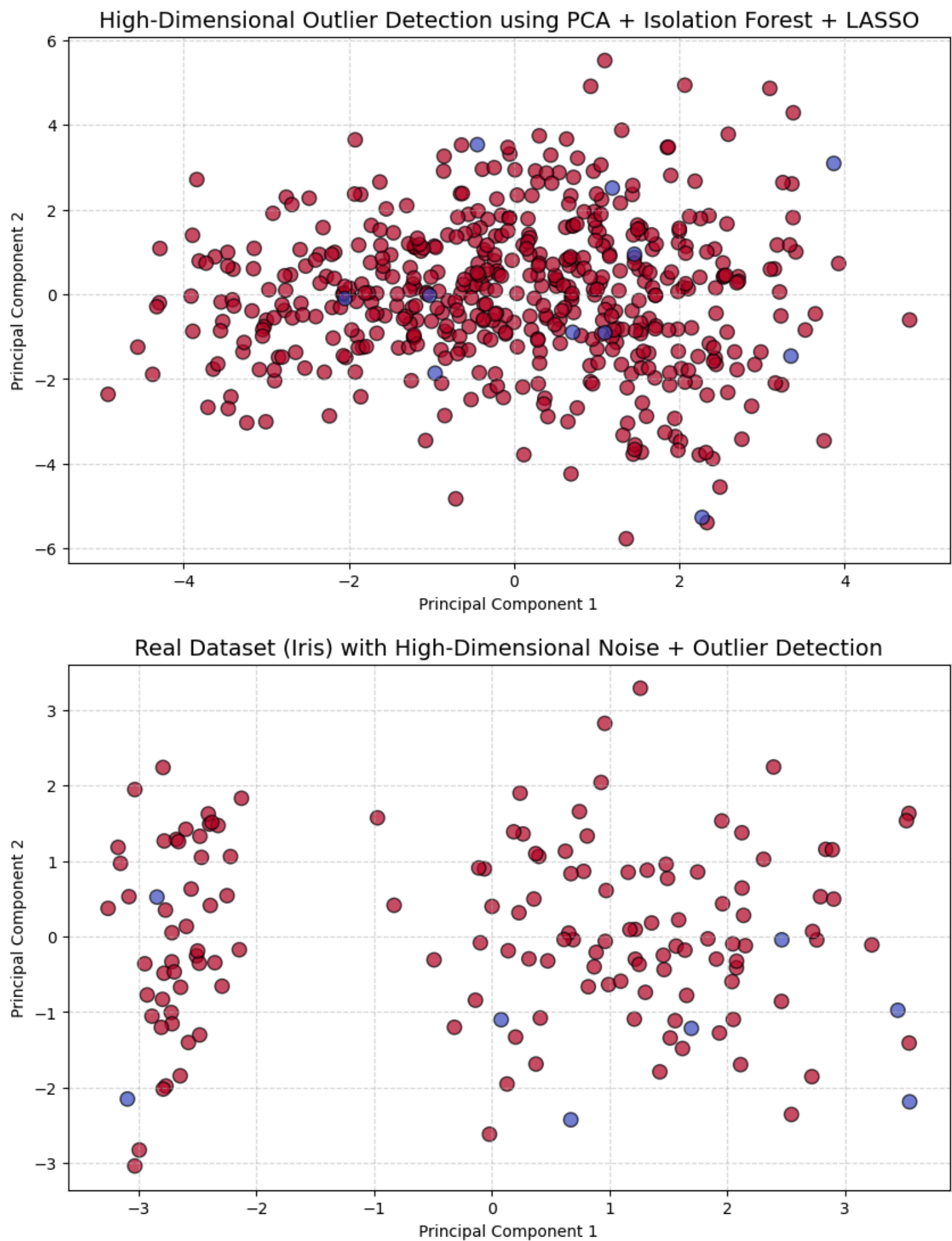
```
iris_data = iris.data
random_noise = np.random.normal(0, 1, size=(iris_data.shape[0], 10))
iris_high_dim = np.hstack([iris_data, random_noise])

# کاهش بعد با PCA
pca_iris = PCA(n_components=2)
iris_pca = pca_iris.fit_transform(iris_high_dim)

# تشخیص نقاط پرت با Isolation Forest
iso_iris = IsolationForest(contamination=0.05, random_state=42)
labels_iris = iso_iris.fit_predict(iris_high_dim)

# نمایش تصویری
plt.figure(figsize=(10, 6))
plt.scatter(iris_pca[:, 0], iris_pca[:, 1], c=labels_iris, cmap='coolwarm', alpha=0.7, s=80, edgecolor='k')
plt.title('Real Dataset (Iris) with High-Dimensional Noise + Outlier Detection', fontsize=14)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

از 20 5 ویژگی‌های انتخاب‌شده توسط LASSO:



توضیحات مثال کد پایتون

در این مثال، از ترکیب سه روش قدرتمند برای تشخیص نقاط پرت در داده‌های با ابعاد بالا استفاده شده است:

- مدل **LASSO** برای انتخاب ویژگی‌های مهم و حذف ویژگی‌های غیرضروری استفاده شده است.

- با استفاده از **PCA** داده‌ها به فضای دوبعدی نگاشته شده‌اند تا امکان تصویرسازی فراهم شود.
- الگوریتم **Isolation Forest** برای تشخیص نقاط پرت اجرا شده و برچسب‌های -1 و 1 تولید کرده است.
- در بخش دوم، همین رویکرد روی داده واقعی **Iris** با ویژگی‌های نویزی اضافه‌شده اعمال شده است تا کاربرد عملی آن نشان داده شود.

این ترکیب برای مسائل واقعی مانند تشخیص تقلب، تحلیل رفتار کاربران، یا پایش سلامت سیستم‌ها بسیار مؤثر است. استفاده از مدل‌های پراکنده به کاهش پیچیدگی و افزایش تفسیرپذیری کمک می‌کند.

11.7.2. Finding outliers in subspaces

📖 یافتن داده‌های پرت در زیرفضاها (Finding outliers in subspaces)

یافتن داده‌های پرت (outliers) در زیرفضاها (subspaces) شامل شناسایی ناهنجاری‌ها در زیرمجموعه‌های خاصی از ویژگی‌ها به جای در نظر گرفتن کل فضای با ابعاد بالا است. این رویکرد تشخیص می‌دهد که داده‌های پرت ممکن است فقط در ترکیبات خاصی از ویژگی‌ها ظاهر شوند و تکنیک‌های سنتی تشخیص داده پرت ممکن است چنین ناهنجاری‌های محلی را ثبت نکنند.

نکات کلیدی درباره یافتن داده‌های پرت در زیرفضاها:

1. **ناهنجاری‌های محلی (Localized Anomalies):** داده‌های پرت ممکن است در زیرفضاها یا ترکیبات خاصی از ویژگی‌ها وجود داشته باشند و این ناهنجاری‌ها ممکن است هنگام تحلیل کل مجموعه ویژگی‌ها به طور همزمان آشکار نباشند.
2. **کاوش زیرفضا (Subspace Exploration):** تکنیک‌ها شامل کاوش زیرمجموعه‌های ویژگی یا زیرفضاهای مختلف برای شناسایی اینکه کدام ترکیبات از ویژگی‌ها به بروز داده‌های پرت کمک می‌کنند، می‌باشند.
3. **رویکردهای ترکیبی (Combinatorial Approaches):** روش‌های ترکیبی، مانند الگوریتم‌های انتخاب ویژگی یا کاوش بَـزورانه (brute-force) ترکیبات ویژگی، می‌توانند به شناسایی زیرفضاهای مرتبط برای تشخیص داده پرت کمک کنند.
4. **نفرین ابعاد (Curse of Dimensionality):** در فضاها با ابعاد بالا، داده‌ها ممکن است به‌طور یکنواخت پخش شوند و تشخیص ناهنجاری‌ها دشوار شود. تمرکز بر زیرفضاها می‌تواند این مشکل را کاهش دهد.
5. **کاربردها (Applications):** این رویکرد برای مجموعه‌های داده‌ای که داده‌های پرت در زمینه‌ها یا زیرمجموعه‌های خاصی از ویژگی‌ها برجسته هستند، مفید است، مانند شبکه‌های حسگر، تشخیص‌های پزشکی، یا هر حوزه‌ای که در آن ترکیبات مختلفی از ویژگی‌ها ممکن است رفتار ناهنجار را نشان دهند.

اکنون، بیا یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای یافتن داده‌های پرت در زیرفضاها با استفاده از یک رویکرد ترکیبی ارائه دهیم:

In []: *Isolation Forest آموزش: تشخیص نقاط پرت در زیرفضاهای تصادفی با استفاده از #*

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.ensemble import IsolationForest
from itertools import combinations
import random
```

```
# تولید داده‌های مصنوعی با ۳۰ ویژگی (۱۰ اطلاع‌رسان، ۵ افزونه)
data, _ = make_classification(
    n_samples=500,
    n_features=20,
    n_informative=10,
    n_redundant=5,
    n_clusters_per_class=1,
    random_state=42
```

```
)

# تولید ۱۰ نقطه پرت با پراکندگی بالا
outliers = np.random.normal(loc=0, scale=5, size=(10, 20))

# ترکیب داده‌های نرمال و پرت
data = np.vstack([data, outliers])

# تعریف تعداد ویژگی‌های انتخاب‌شده در هر زیرفضا
num_features_per_subspace = 3

# تولید تمام ترکیب‌های ممکن از ویژگی‌ها و نمونه‌گیری تصادفی از آنها
all_feature_combinations = list(combinations(range(data.shape[1]), num_features_per_subspace))
feature_combinations = random.sample(all_feature_combinations, 300)

# در هر زیرفضا و شمارش دفعات تشخیص پرت بودن
outliers_subspaces = np.zeros(data.shape[0])
for combination in feature_combinations:
    data_subspace = data[:, combination]
    isolation_forest = IsolationForest(
        contamination=0.02,
        n_estimators=50,
        max_samples=100,
        random_state=42
    )
    labels = isolation_forest.fit_predict(data_subspace)
    outliers_subspaces[labels == -1] += 1

# تعیین آستانه برای تشخیص نهایی نقاط پرت
threshold = 3
outliers_final = outliers_subspaces >= threshold

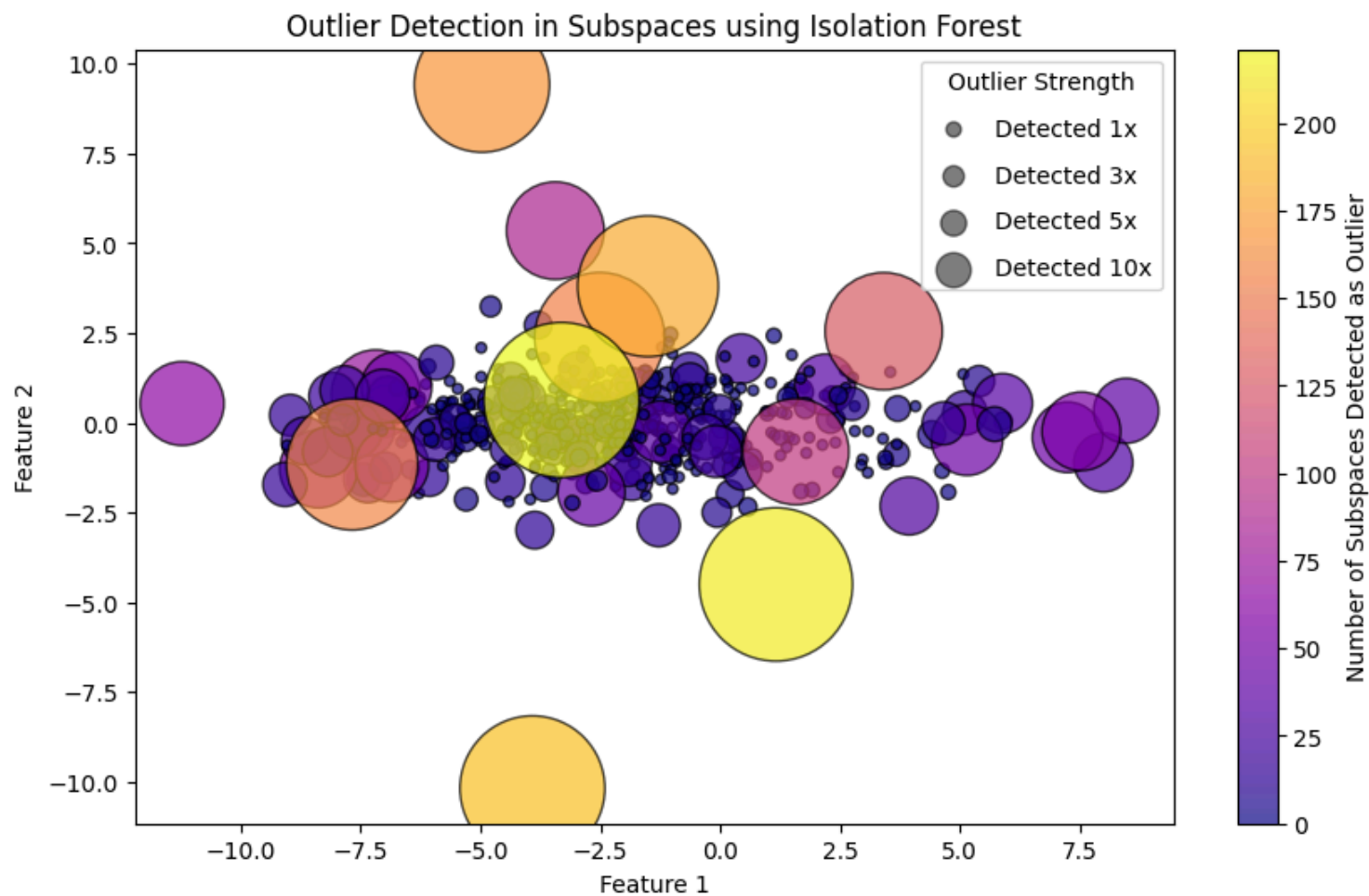
# نمایش تصویری: رنگ و اندازه متناسب با قدرت تشخیص در زیرفضاها
plt.figure(figsize=(10, 6))
sc = plt.scatter(
    data[:, 0], data[:, 1],
    c=outliers_subspaces,
    s=20 + outliers_subspaces*20,
    cmap='plasma', alpha=0.7, edgecolor="k"
)

plt.title('Outlier Detection in Subspaces using Isolation Forest')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

# برای نمایش شدت پرت بودن Legend افزودن
for count in [1, 3, 5, 10]:
    plt.scatter([], [], c='k', alpha=0.5, s=20 + count*20,
                label=f'Detected {count}x')
plt.legend(scatterpoints=1, frameon=True, labelspace=1, title="Outlier Strength")

# برای نمایش تعداد دفعات تشخیص colorbar افزودن
cbar = plt.colorbar(sc)
cbar.set_label('Number of Subspaces Detected as Outlier')

plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از رویکرد تشخیص نقاط پرت در زیرفضاهای تصادفی استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با ابعاد بالا تولید شده‌اند و نقاط پرت به‌صورت دستی اضافه شده‌اند.
- تعداد زیادی زیرفضا با انتخاب تصادفی ۳ ویژگی از بین ۲۰ ویژگی ساخته شده‌اند.
- در هر زیرفضا، الگوریتم **Isolation Forest** اجرا شده و دفعاتی که هر نقطه به‌عنوان پرت تشخیص داده شده، شمارش شده‌اند.
- نقاطی که در بیش از ۳ زیرفضا پرت تشخیص داده شده‌اند، به‌عنوان پرت نهایی در نظر گرفته شده‌اند.
- در نمودار نهایی، رنگ و اندازه هر نقطه نشان‌دهنده قدرت پرت بودن آن در زیرفضاهای مختلف است.

این روش برای تشخیص ناهنجاری در داده‌های پیچیده و با ابعاد بالا بسیار مؤثر است، به‌ویژه زمانی که ناهنجاری‌ها فقط در برخی زیرفضاها قابل مشاهده باشند. ترکیب یادگیری ماشین با تحلیل زیرفضاها می‌تواند در مسائل واقعی مانند تشخیص تقلب، تحلیل رفتار کاربران، یا پایش سیستم‌های پیچیده کاربرد داشته باشد.

11.7.3. Outlier detection ensemble

روش‌های ترکیبی (Ensemble methods) در تشخیص داده پرت 🍌

روش‌های ترکیبی در تشخیص داده پرت (Outlier Detection Ensemble Methods) شامل ترکیب خروجی‌های چندین مدل تشخیص داده پرت برای افزایش دقت و استحکام کلی تشخیص داده پرت در داده‌های با ابعاد بالا (High-dimensional Data) است. این رویکرد با هدف کاهش محدودیت‌های مدل‌های فردی و ارائه شناسایی قابل‌اعتمادتر از ناهنجاری‌ها صورت می‌گیرد.

نکات کلیدی درباره روش‌های ترکیبی در تشخیص داده پرت:

- **مدل‌های متنوع (Diverse Models):** روش‌های ترکیبی از نقاط قوت الگوریتم‌های متنوع تشخیص داده پرت بهره می‌برند. هر مدل ممکن است در جنبه‌های مختلفی عالی باشد یا در زیرفضاهای خاصی مؤثرتر عمل کند.
- **استراتژی‌های تجمیع (Aggregation Strategies):** خروجی‌های مدل‌های فردی تجمیع می‌شوند تا یک تصمیم جمعی درباره اینکه آیا یک نقطه داده، یک داده پرت است یا خیر، گرفته شود. استراتژی‌های تجمیع رایج شامل رأی‌گیری (Voting)، میانگین‌گیری (Averaging) یا استفاده از فرامدل‌ها (Meta-models) است.
- **استحکام (Robustness):** روش‌های ترکیبی با کاهش تأثیر ضعف‌ها یا سوگیری‌های مدل فردی، استحکام تشخیص داده پرت را افزایش می‌دهند. این امر به ویژه در داده‌های با ابعاد بالا (High-dimensional Data) که اثربخشی یک مدل واحد ممکن است محدود باشد، حیاتی است.
- **کاربردها (Applications):** در سناریوهایی که داده‌های با ابعاد بالا برای مدل‌های فردی چالش‌برانگیز هستند، مفید است. کاربردها شامل تشخیص تقلب (Fraud Detection)، امنیت شبکه (Network Security) و هر حوزه‌ای است که در آن داده‌های پرت ممکن است الگوهای متنوعی از خود نشان دهند.

اکنون، بیا یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای روش‌های ترکیبی در تشخیص داده پرت با

استفاده از ترکیبی از Isolation Forest و Local Outlier Factor (LOF) ارائه دهیم:

In []: در داده‌های با ابعاد بالا LOF و Isolation Forest آموزش: تشخیص نقاط پرت با ترکیب #

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
```

```
# 1 تولید داده‌های مصنوعی با ۳۰ ویژگی (۱۰ اطلاع‌رسان، ۵ افزونه)
data, _ = make_classification(n_samples=500, n_features=20, n_informative=10,
                             n_redundant=5, n_clusters_per_class=1, random_state=42)

# 2 تولید ۱۰ نقطه پرت با پراکندگی بالا
outliers = np.random.normal(loc=0, scale=6, size=(10, 20))
data = np.vstack([data, outliers])

# 3 برای تصویرسازی PCA کاهش ابعاد با
pca = PCA(n_components=2, random_state=42)
data_pca = pca.fit_transform(data)

# 4 و استخراج نمره پرت بودن Isolation Forest اجرای
iso = IsolationForest(contamination=0.02, random_state=42)
iso_labels = iso.fit_predict(data)
iso_scores = -iso.decision_function(data) # (بزرگ‌تر → پرت‌تر) نمره پرت بودن

# 5 و استخراج نمره پرت بودن Local Outlier Factor اجرای
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.02)
lof_labels = lof.fit_predict(data)
lof_scores = -lof.negative_outlier_factor_

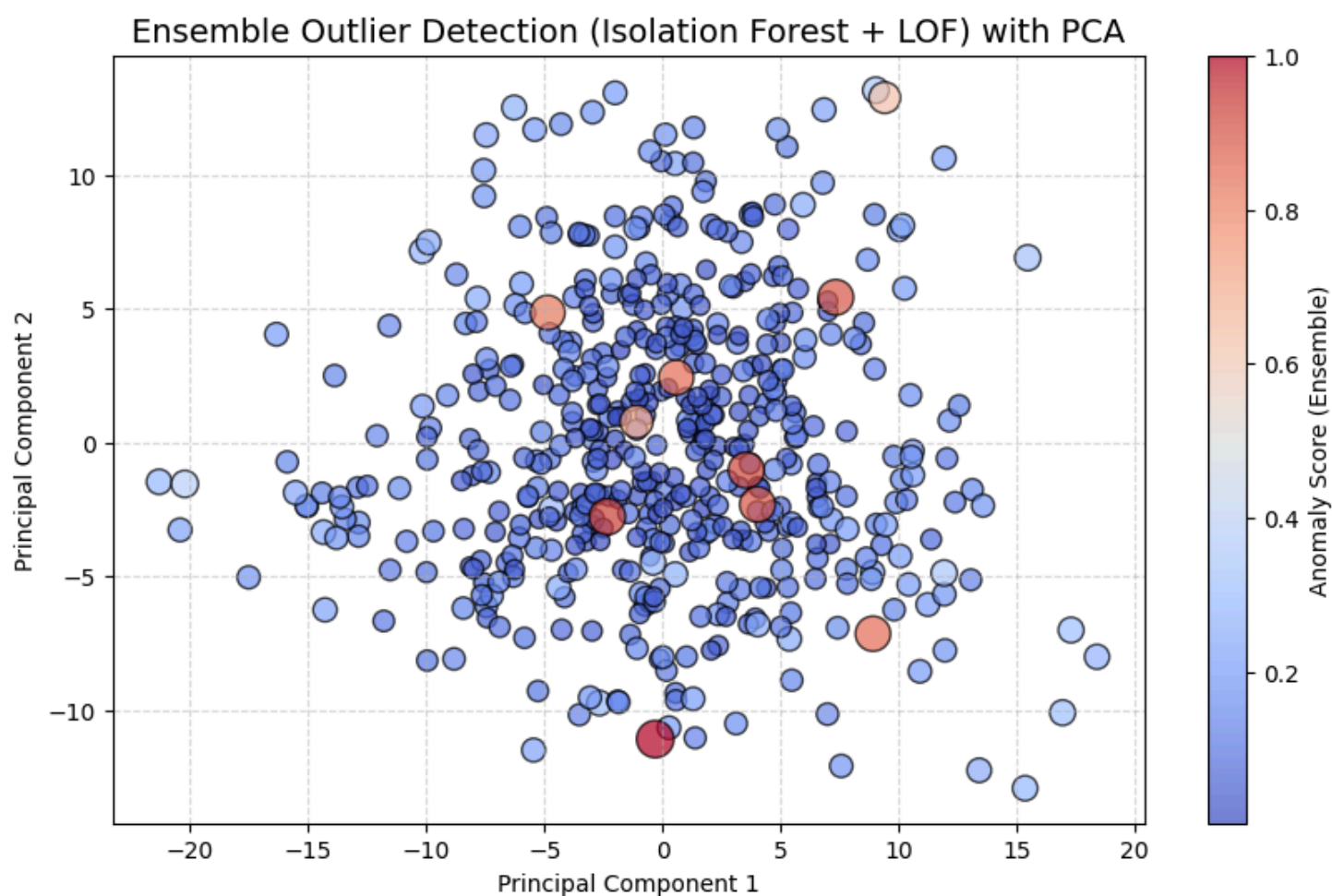
# 6 نرمال‌سازی نمرات برای ترکیب
iso_scores = (iso_scores - iso_scores.min()) / (iso_scores.max() - iso_scores.min())
lof_scores = (lof_scores - lof_scores.min()) / (lof_scores.max() - lof_scores.min())

# 7 ترکیب نمرات با میانگین‌گیری
ensemble_scores = (iso_scores + lof_scores) / 2

# 8 تعیین برچسب نهایی بر اساس آستانه (۲٪ بالاترین نمرات)
threshold = np.percentile(ensemble_scores, 98)
labels = np.where(ensemble_scores >= threshold, -1, 1)

# 9 با رنگ و اندازه متناسب با شدت پرت بودن PCA نمایش تصویری داده‌ها در فضای
plt.figure(figsize=(10, 6))
plt.scatter(data_pca[:, 0], data_pca[:, 1],
            c=ensemble_scores, cmap="coolwarm",
            s=60 + 200 * ensemble_scores,
            edgecolor='k', alpha=0.7)

plt.title("Ensemble Outlier Detection (Isolation Forest + LOF) with PCA", fontsize=14)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(label="Anomaly Score (Ensemble)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از ترکیب دو الگوریتم قدرتمند برای تشخیص نقاط پرت در داده‌های با ابعاد بالا استفاده شده است:

- الگوریتم **Isolation Forest** بر اساس جداسازی تصادفی نمونه‌ها عمل می‌کند و برای داده‌های بزرگ و پیچیده بسیار مؤثر است.
- الگوریتم **Local Outlier Factor (LOF)** با مقایسه چگالی محلی هر نقطه نسبت به همسایگانش، نقاط پرت را شناسایی می‌کند.

- نمرات هر دو الگوریتم نرمال‌سازی شده و با میانگین‌گیری ترکیب شده‌اند تا یک نمره نهایی برای هر نقطه به‌دست آید.
- با استفاده از **PCA** داده‌ها به فضای دوبعدی نگاشته شده‌اند تا امکان تصویرسازی فراهم شود.
- در نمودار نهایی، رنگ و اندازه هر نقطه نشان‌دهنده شدت پرت بودن آن است.

این رویکرد ترکیبی برای تشخیص ناهنجاری در داده‌های واقعی مانند تراکنش‌های مالی، داده‌های حسگر، یا تحلیل رفتار کاربران بسیار مؤثر است. ترکیب چند الگوریتم می‌تواند دقت و پایداری تشخیص را افزایش دهد.

11.7.4. Taming high dimensionality by deep learning

🧠 مهار ابعاد بالا (High dimensionality) با یادگیری عمیق (Deep learning)

مهار ابعاد بالا با یادگیری عمیق شامل بهره‌گیری از معماری‌های شبکه عصبی (neural network architectures) برای یادگیری خودکار نمایش‌ها و الگوها در داده‌های با ابعاد بالا است. مدل‌های یادگیری عمیق، به ویژه خودرمزگذارها (autoencoders)، برای کاهش ابعاد، یادگیری ویژگی (feature learning) و ثبت ساختارهای پیچیده در داده‌ها به کار گرفته می‌شوند که به شناسایی داده‌های پرت (outliers) کمک می‌کند.

نکات کلیدی درباره مهار ابعاد بالا با یادگیری عمیق:

- **خودرمزگذارها (Autoencoders):** خودرمزگذارها معماری‌های شبکه عصبی هستند که برای وظایف یادگیری بدون نظارت (unsupervised learning) از جمله کاهش ابعاد طراحی شده‌اند. آن‌ها از یک رمزگذار (encoder) و یک رمزگشا (decoder) تشکیل شده‌اند، که رمزگذار داده ورودی را به یک نمایش با ابعاد پایین‌تر نگاشت می‌کند و رمزگشا ورودی را از این نمایش بازسازی می‌نماید.
- **یادگیری ویژگی (Feature Learning):** مدل‌های یادگیری عمیق، با یادگیری نمایش‌های سلسله‌مراتبی، می‌توانند ویژگی‌ها و روابط پیچیده را در داده‌های با ابعاد بالا ثبت کنند. این امر با تمرکز بر الگوهای مرتبط، تشخیص مؤثرتر داده پرت را ممکن می‌سازد.
- **تبدیلات غیرخطی (Nonlinear Transformations):** مدل‌های یادگیری عمیق قادر به ثبت روابط غیرخطی در داده‌ها هستند، که به ویژه در فضاهای با ابعاد بالا که روش‌های خطی ممکن است کارایی نداشته باشند، ارزشمند است.
- **نفرین ابعاد (Curse of Dimensionality):** یادگیری عمیق با کاهش مؤثر ابعاد و تمرکز بر ویژگی‌های مهم، می‌تواند اثرات منفی نفرین ابعاد را کاهش دهد و تحلیل داده‌های پرت را بهبود بخشد.
- **کاربردها (Applications):** مهار ابعاد بالا با یادگیری عمیق در حوزه‌های مختلفی از جمله پردازش تصویر، ژنومیک و هر رشته‌ای که با داده‌های پیچیده و با ابعاد بالا سروکار دارد و روش‌های سنتی ممکن است در آن دچار مشکل شوند، مفید است.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای مهار ابعاد بالا با یادگیری عمیق با استفاده از یک خودرمزگذار ارائه دهیم:

In [3]: Autoencoder آموزش: تشخیص نقاط پرت در داده‌های با ابعاد بالا با استفاده از #

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

تولید داده‌های مصنوعی با ۲۰ ویژگی (۱۰ اطلاع‌رسان، ۵ افزونه) 1

```
data, _ = make_classification(
    n_samples=500,
    n_features=20,
    n_informative=10,
    n_redundant=5,
    n_clusters_per_class=1,
    random_state=42
)

# تولید ۱۰ نقطه پرت با پراکندگی بالا
outliers = np.random.normal(loc=0, scale=5, size=(10, 20))
data = np.vstack([data, outliers])

# استانداردسازی داده‌ها برای آموزش پایدار شبکه
scaler = StandardScaler()
data_standardized = scaler.fit_transform(data)

# تعریف و آموزش مدل Autoencoder
input_dim = data_standardized.shape[1]
model = Sequential([
    Dense(15, activation='relu', input_dim=input_dim),
    Dense(8, activation='relu'),
    Dense(15, activation='relu'),
    Dense(input_dim, activation='linear')
])
model.compile(optimizer='adam', loss='mean_squared_error')
history = model.fit(
    data_standardized, data_standardized,
    epochs=50, batch_size=32,
    shuffle=True, validation_split=0.2,
    verbose=0
)

# بازسازی داده‌ها و محاسبه خطای بازسازی برای هر نمونه
decoded_data = model.predict(data_standardized)
mse = np.mean(np.square(data_standardized - decoded_data), axis=1)

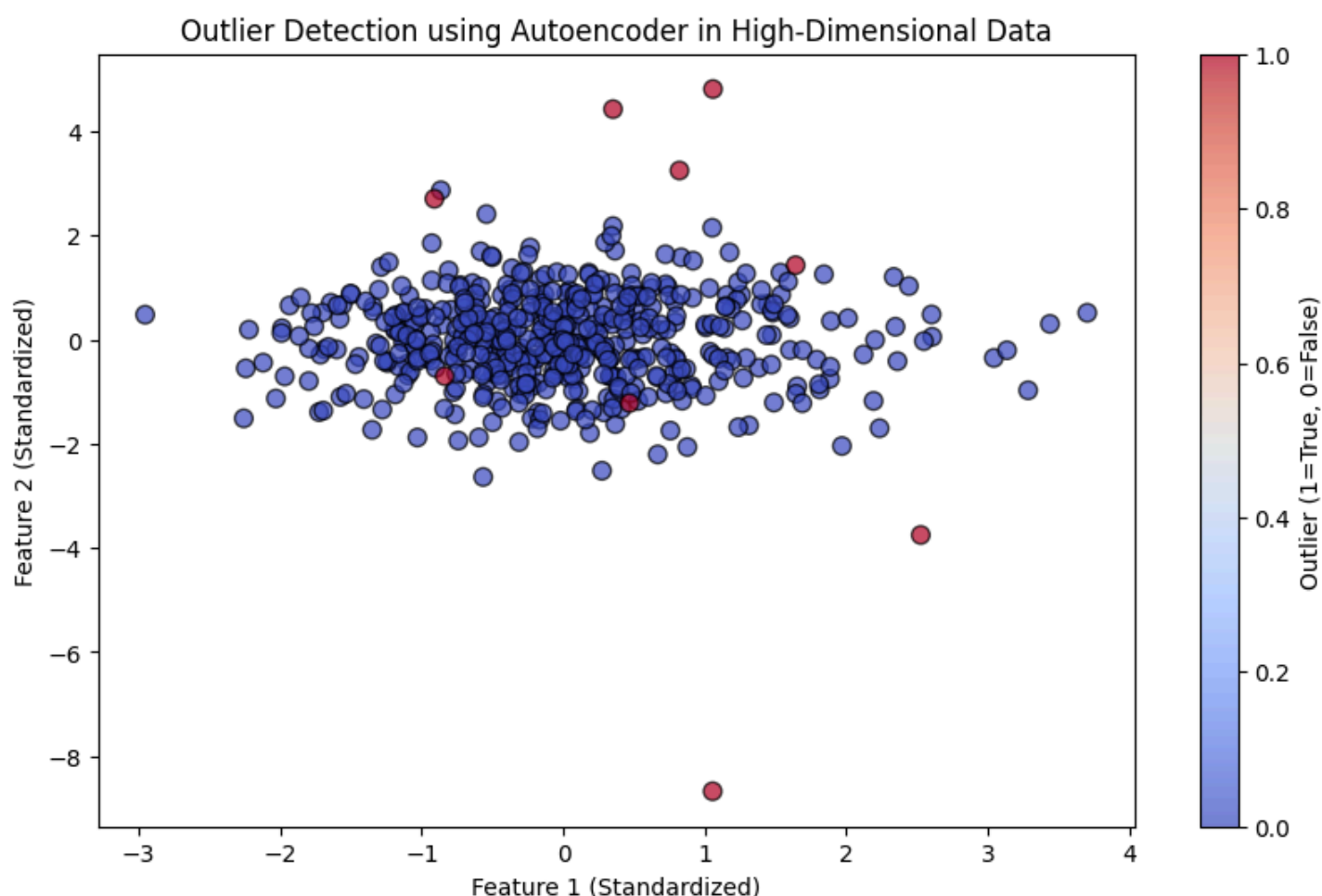
# تعیین آستانه برای تشخیص نقاط پرت (میانگین + ۲ انحراف معیار)
threshold = np.mean(mse) + 2*np.std(mse)
outliers = mse > threshold

# نمایش تصویری داده‌ها با رنگ‌بندی بر اساس پرت بودن
plt.figure(figsize=(10, 6))
plt.scatter(data_standardized[:, 0], data_standardized[:, 1],
            c=outliers, cmap='coolwarm', alpha=0.7, s=60, edgecolor='k')
plt.title('Outlier Detection using Autoencoder in High-Dimensional Data')
plt.xlabel('Feature 1 (Standardized)')
plt.ylabel('Feature 2 (Standardized)')
plt.colorbar(label="Outlier (1=True, 0=False)")
plt.show()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

16/16 ————— 1s 19ms/step



توضیحات مثال کد پایتون

در این مثال، از مدل **Autoencoder** برای تشخیص نقاط پرت در داده‌های با ابعاد بالا استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با ویژگی‌های اطلاع‌رسان و افزونه تولید شده‌اند و نقاط پرت به‌صورت دستی اضافه شده‌اند.
- داده‌ها با استفاده از `StandardScaler` استانداردسازی شده‌اند تا آموزش شبکه پایدارتر باشد.
- مدل `Autoencoder` با ساختار ساده آموزش دیده تا داده‌ها را بازسازی کند.
- برای هر نمونه، خطای بازسازی محاسبه شده و نمونه‌هایی با خطای بالا به‌عنوان نقاط پرت شناسایی شده‌اند.
- نتایج با استفاده از نمودار پراکندگی نمایش داده شده‌اند تا نقاط پرت به‌صورت رنگی قابل تشخیص باشند.

مدل‌های `Autoencoder` برای تشخیص ناهنجاری در داده‌های پیچیده و بدون برچسب بسیار مؤثر هستند. این روش به‌ویژه در مسائل واقعی مانند تشخیص تقلب، پایش سلامت تجهیزات، و تحلیل رفتار کاربران کاربرد دارد.

11.7.5. Modeling high-dimensional outliers

مدل‌سازی داده‌های پرت با ابعاد بالا (Modeling High-Dimensional Outliers)

مدل‌سازی داده‌های پرت با ابعاد بالا (High-Dimensional Outliers) شامل استفاده از مدل‌های آماری و یادگیری ماشین است که به طور خاص برای ثبت الگوهای پیچیده موجود در داده‌های با ابعاد بالا طراحی شده‌اند. این مدل‌ها با هدف تمایز بین نمونه‌های عادی و ناهنجار (Anomalous Instances) به شیوه‌ای مؤثر برای مجموعه‌های داده با تعداد زیادی ویژگی، صورت می‌گیرد.

نکات کلیدی درباره مدل‌سازی داده‌های پرت با ابعاد بالا:

- **فرضیات توزیعی (Distributional Assumptions):** برخی مدل‌ها یک توزیع خاص از داده‌های عادی را فرض می‌کنند و داده‌های پرت را به عنوان نمونه‌هایی شناسایی می‌کنند که به طور قابل توجهی از این توزیع فرضی منحرف می‌شوند. با این حال، در فضاهای با ابعاد بالا، انتخاب یک توزیع مناسب چالش‌برانگیز می‌شود.
- **معیارهای فاصله (Distance Metrics):** روش‌های مبتنی بر فاصله، مانند فاصله مالهالانوبیس (Mahalanobis Distance)، برای در نظر گرفتن همبستگی‌ها در میان ویژگی‌ها در مجموعه‌های داده با ابعاد بالا تطبیق داده شده‌اند. این روش‌ها فاصله یک نقطه داده از مرکز توزیع را با در نظر گرفتن ساختار کوواریانس (Covariance) اندازه‌گیری می‌کنند.
- **تخمین‌گرهای مقاوم (Robust Estimators):** تخمین‌گرهای آماری مقاوم، مانند تعیین‌کننده کوواریانس حداقل (Minimum Covariance Determinant - MCD)، طوری طراحی شده‌اند که حساسیت کمتری نسبت به داده‌های پرت داشته باشند. آن‌ها می‌توانند به ویژه در سناریوهای با ابعاد بالا که حضور داده‌های پرت می‌تواند تخمین‌گرهای سنتی را مخدوش کند، مفید باشند.
- **مدل‌های یادگیری ماشین (Machine Learning Models):** روش‌های ترکیبی (Ensemble Methods)، ماشین‌های بردار پشتیبان (Support Vector Machines - SVM) و سایر مدل‌های یادگیری ماشین می‌توانند برای تشخیص داده پرت با ابعاد بالا، با در نظر گرفتن چالش‌های ناشی از پدیده نفرین ابعاد (Curse of Dimensionality)، تطبیق داده یا تخصصی شوند.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای مدل‌سازی داده‌های پرت با ابعاد بالا با استفاده از تخمین‌گر تعیین‌کننده کوواریانس حداقل (Minimum Covariance Determinant - MCD) ارائه دهیم:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler
from sklearn.covariance import EllipticEnvelope
from scipy.spatial.distance import mahalanobis
from sklearn.decomposition import PCA

# 1 تولید داده‌های مصنوعی با ۲۰ ویژگی (۱۰ اطلاع‌رسان، ۵ افزونه)
data, _ = make_classification(
    n_samples=500,
    n_features=20,
    n_informative=10,
    n_redundant=5,
    n_clusters_per_class=1,
    random_state=42
)

# 2 تولید ۱۰ نقطه پرت با پراکندگی بالا
outliers = np.random.normal(loc=0, scale=5, size=(10, 20))
data = np.vstack([data, outliers])

# 3 استانداردسازی داده‌ها برای پایداری آماری
scaler = StandardScaler()
data_std = scaler.fit_transform(data)

# 4 برای تثبیت تخمین کوواریانس PCA کاهش ابعاد با
pca = PCA(n_components=10)
data_pca = pca.fit_transform(data_std)

# 5 برای تخمین کوواریانس مقاوم MCD اعمال الگوریتم
mcd = EllipticEnvelope(contamination=0.02, support_fraction=0.95, random_state=42)
mcd.fit(data_pca)
labels = mcd.predict(data_pca) # (پرت) 1 و (نرمال) 0

# 6 برای هر نمونه Mahalanobis محاسبه فاصله
mean = mcd.location_
cov = mcd.covariance_
inv_cov = np.linalg.pinv(cov)
mahal_dist = np.array([mahalanobis(x, mean, inv_cov) for x in data_pca])

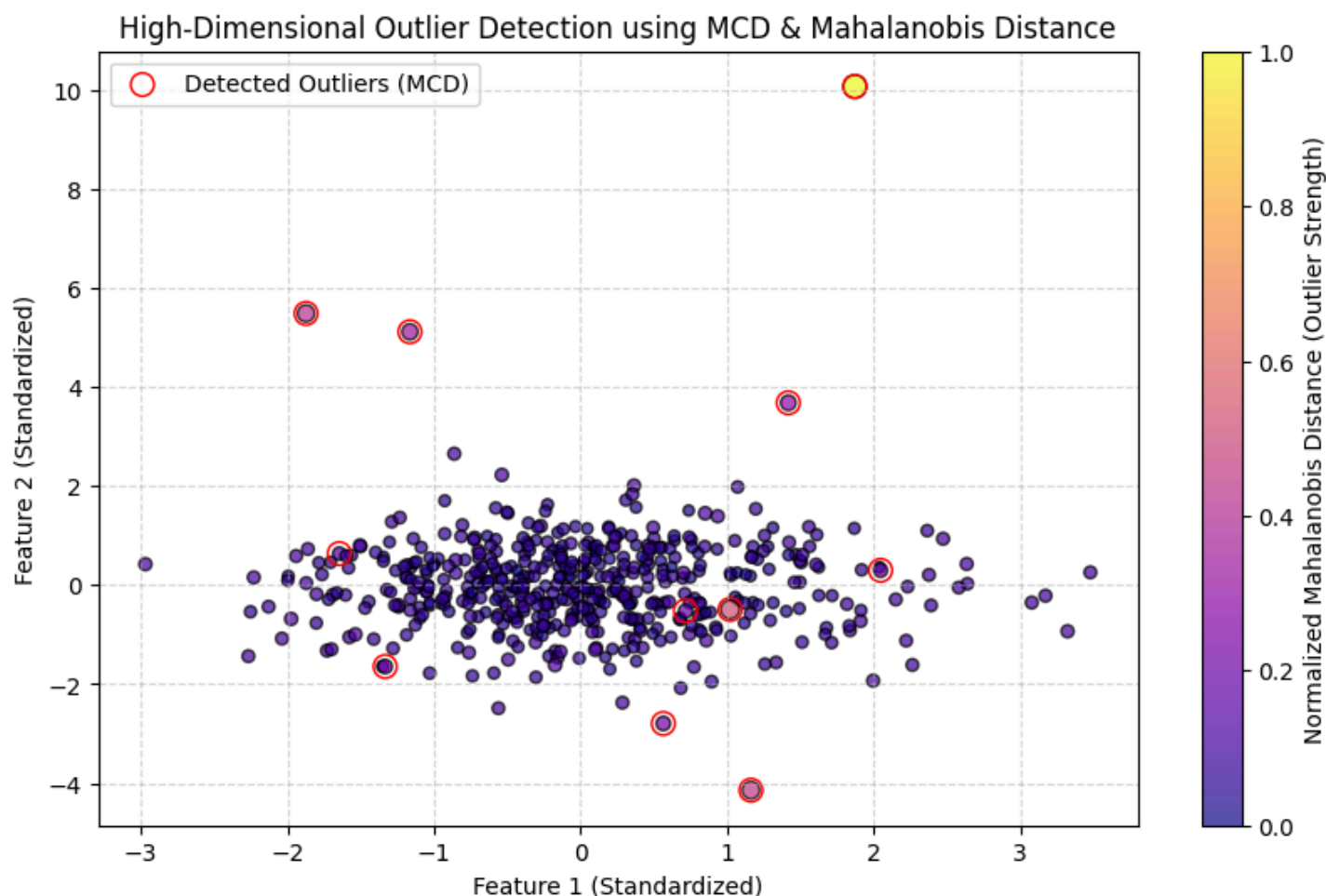
# 7 نرمال‌سازی فاصله‌ها برای نمایش تصویری
mahal_dist_norm = (mahal_dist - mahal_dist.min()) / (mahal_dist.max() - mahal_dist.min())

# 8 نمایش تصویری داده‌ها با رنگ و اندازه متناسب با شدت پرت بودن
plt.figure(figsize=(10, 6))
sc = plt.scatter(
    data_std[:, 0], data_std[:, 1],
    c=mahal_dist_norm,
    s=20 + mahal_dist_norm*80,
    cmap='plasma', alpha=0.7, edgecolor='k'
)

plt.title('High-Dimensional Outlier Detection using MCD & Mahalanobis Distance')
plt.xlabel('Feature 1 (Standardized)')
plt.ylabel('Feature 2 (Standardized)')

# 9 و برجسته‌سازی نقاط پرت colorbar افزودن
cbar = plt.colorbar(sc)
cbar.set_label('Normalized Mahalanobis Distance (Outlier Strength)')
plt.scatter(
    data_std[labels==-1, 0],
    data_std[labels==-1, 1],
    facecolors='none', edgecolors='red', s=100,
    label='Detected Outliers (MCD)'
)

plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از روش آماری **Minimum Covariance Determinant (MCD)** برای تشخیص نقاط پرت در داده‌های با ابعاد بالا استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با ویژگی‌های اطلاع‌رسان و افزونه تولید شده‌اند و نقاط پرت به‌صورت دستی اضافه شده‌اند.
- داده‌ها استانداردسازی شده‌اند و سپس با استفاده از **PCA** به ۱۰ مؤلفه اصلی کاهش یافته‌اند تا تخمین کوواریانس پایدارتر شود.
- مدل **EllipticEnvelope** با استفاده از **MCD** آموزش دیده و نقاط پرت را با برچسب 1- شناسایی کرده است.
- برای هر نمونه، فاصله **Mahalanobis** نسبت به مرکز داده محاسبه شده و نرمال‌سازی شده است.
- در نمودار نهایی، رنگ و اندازه هر نقطه نشان‌دهنده شدت پرت بودن آن است و نقاط پرت با حاشیه قرمز برجسته شده‌اند.

روش **MCD** برای تشخیص ناهنجاری‌های آماری در داده‌های چندبعدی بسیار مناسب است، به‌ویژه زمانی که داده‌ها شامل نویز یا نقاط پرت شدید باشند. ترکیب آن با فاصله **Mahalanobis** و کاهش بعد می‌تواند در مسائل واقعی مانند تشخیص تقلب، تحلیل داده‌های مالی، یا پایش کیفیت داده‌ها کاربرد داشته باشد.

The End of Chapter 11